

智能体实训 @2026

大模型应用实训报告

(DeepSeek Harness 与国内大模型 API 应用)

姓 名 陈昊、胡宇星

学 号 14236202061、14236202065

群 名 智能体实训 @2026

分 工 陈昊：章节代码与实验；胡宇星：教程盲测与文档编写

提交日期 2026-09-02

实训周期：2026-08-20 至 2026-09-02

目录

1 第一章：国内大模型 API 基础实验教程	2
1.1 背景	2
1.2 环境	2
1.3 实验步骤	4
1.3.1 实验 1: API key 与环境变量	4
1.3.2 实验 2: curl 调用	4
1.3.3 实验 3: Python 统一调用脚本	5
1.3.4 实验 4: 流式输出	6
1.3.5 实验 5: JSON 输出	6
1.3.6 实验 6: 错误处理	6
1.3.7 实验 7: 模型对比	6
1.4 结果	6
1.4.1 实验 1 结果: 环境与 API key 配置	7
1.4.2 实验 2 结果: curl 调用 (Kimi 与 MiMo)	10
1.4.3 实验 3 结果: Python 统一调用	12
1.4.4 实验 4 结果: 流式输出	13
1.4.5 实验 5 结果: JSON 输出与校验	15
1.4.6 实验 6 结果: 错误处理	16
1.4.7 实验 7 结果: 模型对比	17
1.5 对比分析	18
1.6 可复现教程	21
2 第二章：提示词到智能体的演化	23
2.1 背景	23
2.2 概念背景：四层各控制什么	23
2.2.1 为什么 context 不是把资料全部塞进 prompt	24
2.2.2 为什么工具说明也是 prompt/context 的一部分	24
2.2.3 harness 与 framework、IDE、MCP、plugin、sandbox、CI 的 关系	25
2.3 实验设计：同一任务四层递进	25
2.4 实验结果	26

2.4.1	第 1 层：只有一句话	27
2.4.2	第 2 层：加入上下文包	28
2.4.3	第 3 层：Harness（工具 + 脚本 + 测试）	29
2.4.4	第 4 层：Loop agent	30
2.5	Agent 的本质	31
2.6	反思	32
3	第三章：Pi-core 源码与工具调用分析	34
3.1	背景	34
3.2	关键文件与职责	35
3.3	工具调用链路与时序	36
3.3.1	主链路（源码确认，agent-loop.ts）	36
3.3.2	关键函数解释（≥3 个）	37
3.4	错误路径（≥4 条，均有源码依据）	38
3.5	设计评价（≥2 个）	38
3.6	与 DSH 的横向比较	39
4	第四章：DeepSeek Harness 融入具体应用	41
4.1	背景	41
4.2	环境	41
4.3	官方教程复现	43
4.3.1	Your first plugin：创建插件并看到终端输出	43
4.3.2	Build a tool：greet 被模型真实调用	45
4.3.3	业务工具：dev_environment_snapshot（greet 的改造）	46
4.3.4	安全边界（任务书必做实验 5 的设计与分析）	47
4.4	原理分析	48
4.4.1	inject = ['tools'] 为什么必要	48
4.4.2	defineTool 各字段职责	49
4.4.3	工具进入模型上下文的链路	49
4.4.4	profile / bundle / patch 分层	50
4.4.5	远程运行与安全访问（WSL 视作独立远程主机）	50
4.5	问题与改进	53
5	第五章（选作）：用 DSH 实现个人 Alpha ——项目学习报告助手	55

5.1	背景：问题定义	55
5.2	实验目标	55
5.3	步骤与环境	55
5.4	结果：真实运行证据	56
5.4.1	交付文件 1: TECHNICAL_REPORT.md	57
5.4.2	交付文件 2: REPRODUCTION.md	58
5.5	分析与反思	58
A	附录	61
A.1	代码仓库地址	61
A.2	环境版本总表	62
A.3	参考资料总表	62
A.4	截图脱敏声明	62

1 第一章：国内大模型 API 基础实验教程

1.1 背景

大模型 API 本质上是一个带鉴权的 HTTP 服务。客户端通过 POST 请求发送 JSON 数据，Header 中携带 `Content-Type: application/json` 与 `Authorization: Bearer <API_KEY>`，Body 中通常包含 `model`、`messages`、`stream`、`temperature`、`response_format` 或 `tools` 等字段。响应同样是 JSON，常见字段包括 `choices`、`message`、`usage`、`finish_reason` 和错误信息。只有先把“HTTP 请求—JSON 请求体—鉴权—响应解析—错误处理—费用控制”这条链路打牢，后续的 `agent`、工具调用和 DSH 插件才不会变成黑箱。

国内主流平台（火山方舟、Kimi、阿里云百炼）均提供 OpenAI-compatible 接口，同一段 OpenAI SDK 代码通常只需替换 API key、`base_url` 和 `model` 即可切换平台。但不同平台在地域、模型名、Endpoint ID、工具调用、JSON mode、流式 `usage`、错误码和限流策略上仍有差异，本章将逐一记录这些差异。

本章按任务书第一章的 7 个必做实验展开：API key 与环境变量、curl 调用、Python 统一调用、流式输出、JSON 输出、错误处理、模型对比。

1.2 环境

本章全部实验在 Windows 本机完成，运行环境见表 1。Python 使用 Miniconda 的 ML-base 环境（Python 3.11.15），通过 openai SDK（v2.37.0）调用各平台。

表 1 第 1 章实验环境

项目	版本/配置
操作系统	Windows 11 专业版 (build 26200 / 25H2; PowerShell 7 终端)
Python	3.11.15 (Miniconda 环境 ML-base)
openai SDK	2.37.0
matplotlib	3.10.9 (对比图绘制)
Node.js / npm	v24.18.0 / 12.0.2
curl	8.21.0 (libcurl/8.21.0 Schannel)
jq	1.8.2
模型 1	Kimi kimi-k3 (api.moonshot.cn/v1)
模型 2	小米 MiMo mimo-v2.5-pro (api.xiaomimimo.com/v1)
模型 3	Qwen qwen-plus (dashscope.aliyuncs.com/compatible-mode/v1)
模型 4	火山方舟 (ark.cn-beijing.volces.com/api/v3, Model/Endpoint ID)
运行日期	2026-08-18

三个平台的 API key 统一通过环境变量管理 (表 2)，源码中不出现任何明文密钥；报告中的截图与日志一律脱敏。

环境限制说明：本实验在 DSH (DeepSeek Harness) 沙箱环境中完成部分运行。沙箱会拦截 curl (schannel 后端) 访问 Windows 证书凭据存储 (SEC_E_NO_CREDENTIALS)，因此实验 2 的 curl 调用在放宽权限的模式下执行；普通终端 (非沙箱) 运行同一 curl 命令无此限制。沙箱同样会把子进程窗口隔离在不可见窗口站，故实验截图由用户在普通终端手动抓取。若在普通环境下复现，所有命令均可直接运行 (见可复现教程一节)。

表 2 API key 环境变量约定（截图/日志必须脱敏）

平台	环境变量	申请入口	本实验状态
火山方舟	ARK_API_KEY	console.volcengi ne.com/ark	已注册（图 3）
Kimi（月之暗面）	MOONSHOT_API_KEY	platform.moonsho t.cn	已注册并调用（图 1）
小米 MiMo	MIMO_API_KEY	platform.xiaomim imo.com	已注册并调用（图 2）
阿里云百炼	DASHSCOPE_API_KEY	bailian.console. aliyun.com	已注册（图 4）

1.3 实验步骤

1.3.1 实验 1：API key 与环境变量

在 code/ch1-api/.env 中统一配置三个平台的 key（该文件已被 .gitignore 忽略，严禁提交仓库）：

```
ARK_API_KEY=
MOONSHOT_API_KEY=
DASHSCOPE_API_KEY=
```

脚本通过 common.load_env() 读取该文件并写入环境变量，读取时遵循“已存在的环境变量不覆盖”原则，保证 CI/服务器场景下可以直接用系统环境变量注入密钥。密钥在使用与记录时均做脱敏处理（只显示前 6 位与后 4 位）。

1.3.2 实验 2：curl 调用

curl 是最直接的 HTTP 客户端，适合验证“大模型 API 就是一个带鉴权的 HTTP 服务”这一本质。以 Kimi 为例（完整命令见 notes/curl_calls.md）：

```
curl.exe -sS https://api.moonshot.cn/v1/chat/completions `
-H "Content-Type: application/json" `
-H "Authorization: Bearer $env:MOONSHOT_API_KEY" `
-d "{\"model\":\"kimi-k3\",\"messages\":[{\"role\":\"user\",
    \"content\":\"用一句话介绍 HTTP\"}],\"max_tokens\":100}"
```

要点：-H 指定请求头（Content-Type 声明请求体为 JSON，Authorization 携带 Bearer token）；-d 为 JSON 请求体；响应中的 HTTP 状态码是判断结果的第一信号（200 成功、400 请求体错误、401 鉴权失败、404 路径错误、429 限流）。对小米 MiMo 只需替换 base_url（api.xiaomimimo.com/v1）、Authorization 与 model 三项，同样体现了 OpenAI-compatible 的低迁移成本。

1.3.3 实验 3：Python 统一调用脚本

按照任务书给出的骨架，实现 unified_call.py：通过 provider 名称切换 base_url、model 与 api_key，同一问题同时调用两个模型，并输出响应内容与 usage：

```
PROVIDERS = {
    "kimi": {"env": "MOONSHOT_API_KEY",
            "base_url": "https://api.moonshot.cn/v1",
            "model": "kimi-k3"},
    "mimo": {"env": "MIMO_API_KEY",
            "base_url": "https://api.xiaomimimo.com/v1",
            "model": "mimo-v2.5-pro"},
    "ark": {"env": "ARK_API_KEY",
            "base_url": "https://ark.cn-beijing.volces.com/"
                    "api/v3",
            "model": "<Model ID 或 Endpoint ID>"},
}

resp = client.chat.completions.create(
    model=cfg["model"],
    messages=[{"role": "system", "content": ...},
              {"role": "user", "content": prompt}])
print(resp.choices[0].message.content)
print(resp.usage.model_dump())
```


1.3.4 实验 4：流式输出

启用 `stream=True` 后，服务端以 SSE 方式逐个返回 chunk，每个 chunk 的 `delta.content` 字段是增量文本；通过 `stream_options` 参数（设为 `include_usage=True`）可在流末尾拿到 `usage`。流式输出的价值在于首字节延迟（TTFT）远小于完整响应时间，用户可以在生成过程中就看到内容。本章同时做了一次普通（非流式）调用作为对照，比较 TTFT 与总耗时。

1.3.5 实验 5：JSON 输出

通过 `response_format={"type": "json_object"}` 强制模型输出合法 JSON，再让模型按固定 schema 组织内容（课程计划：`title`、`tasks`、`deadline`、`risks`），最后用 Python `json.loads` 解析并逐字段校验。该实验对应真实工程中“模型输出结构化数据供程序消费”的常见需求。

1.3.6 实验 6：错误处理

故意构造四类错误请求并记录错误码与排查过程：

1. 错误 API key（无效密钥），预期 401；
2. 错误模型名（不存在的 model），预期 400/404；
3. 错误 `base_url`（路径后追加 `/badpath`），预期 404；
4. 请求体缺字段（`messages` 缺失），预期 400。

每类错误都记录异常类型、HTTP 状态码与服务端返回的错误信息，并给出排查建议（见实验结果小节）。

1.3.7 实验 7：模型对比

同一中文任务（输出一个 JSON 格式的学习计划）分别调用 Kimi 与 MiMo，比较四项指标：内容质量（响应字数、要点覆盖）、格式遵循（JSON 是否可解析）、速度（TTFT 与总耗时）、费用（`token` 用量 × 官方单价估算）。对比结果以表格与柱状图（图 14）呈现。

1.4 结果

本节给出各实验的真实运行输出。每个实验均保留了两种证据：真实运行窗口截图（图 6 至图 15，2026-08-18 抓取，截图中的 API key 均已脱敏或从未显示）

与 UTF-8 原始日志 (evidence/ch1/ 目录), 两者内容一致、相互印证。

1.4.1 实验 1 结果：环境与 API key 配置

环境检查真实运行窗口见图 6：系统为 Windows 11 专业版 (build 26200 / 25H2, NT 内核版本号为 10.0.26200, platform 模块因此显示 Windows 10), Python 3.11.15、Node.js v24.18.0; MOONSHOT_API_KEY 与 MIMO_API_KEY 已配置 (脱敏显示); ARK_API_KEY、DASHSCOPE_API_KEY 对应平台账号已注册并持有 key (见下方控制台截图), 仅未写入本机 .env (本实验未调用这两个平台)。

API key 由作者提前在平台注册获取 (key 申请即控制台开通, 申请后即可在控制台创建与管理 API key), 四个国内平台的控制台均已注册并持有 API key: Kimi (图 1)、小米 MiMo (图 2)、火山方舟 (图 3)、阿里云百炼 (图 4); 另附 DeepSeek 平台控制台 (图 5) 作为国内 OpenAI-compatible 平台通用界面的补充示例。所有截图均未包含 key 明文 (平台本身以星号/部分字符遮挡); DeepSeek 截图中的充值余额已遮挡, 显示的累计消费金额为账户历史统计; MiMo 控制台显示的累计消费 (¥0.21) 与本实验的费用估算 (表 7) 量级一致, 可互相印证。

实际调用的 provider 选择: 本实验实际调用使用 Kimi 与小米 MiMo 两个 provider (满足任务书”至少两个国内 provider 调通”)。选择主要出于经济考虑: 阿里云百炼 (Qwen) 与火山方舟的新用户免费赠送 token 额度已在使用本实验之前消耗完毕 (火山方舟对新注册用户赠送免费 token 试用额度; 阿里云百炼亦提供新用户免费额度), 继续使用需按量付费充值; 而 Kimi、小米 MiMo 与 DeepSeek 账户仍有可用余额, 故优先调用有余额的 Kimi 与 MiMo 完成实验 (DeepSeek 账户亦有余额, 可作备用 provider)。火山方舟与阿里云百炼的 API key 均已注册 (图 3、图 4), 配置环境变量后即可调用 (见可复现教程)。

项目 ID	模型	时间	备注	API Key ID	API Key ID	输入 Tokens	输出 Tokens	Cached Tokens
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:37:09	default	test	ak-6a7e1f883ae11f0a0e1	438	3794	
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:35:14	default	test	ak-6a7e1f883ae11f0a0e1	443	3475	
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:33:04	default	test	ak-6a7e1f883ae11f0a0e1	399	3602	256
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:28:35	default	test	ak-6a7e1f883ae11f0a0e1	88	224	88
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:28:35	default	test	ak-6a7e1f883ae11f0a0e1	386	3166	256
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:26:56	default	test	ak-6a7e1f883ae11f0a0e1	399	6546	256
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:23:56	default	test	ak-6a7e1f883ae11f0a0e1	98	3146	98
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:23:09	default	test	ak-6a7e1f883ae11f0a0e1	98	2323	98
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:23:06	default	test	ak-6a7e1f883ae11f0a0e1	98	2291	98
chatmpt-6dd9763294148230a7581	kimi-k3	2026-08-19 01:19:21	default	test	ak-6a7e1f883ae11f0a0e1	98	3710	98

图 1 实验 1: Kimi 平台 (platform.moonshot.cn) 控制台计费明细页面——kimi-k3 调用记录与输入/输出 Tokens (2026-08-19 多笔调用; API Key 名称与项目 ID 均已脱敏, 无 key 明文)。

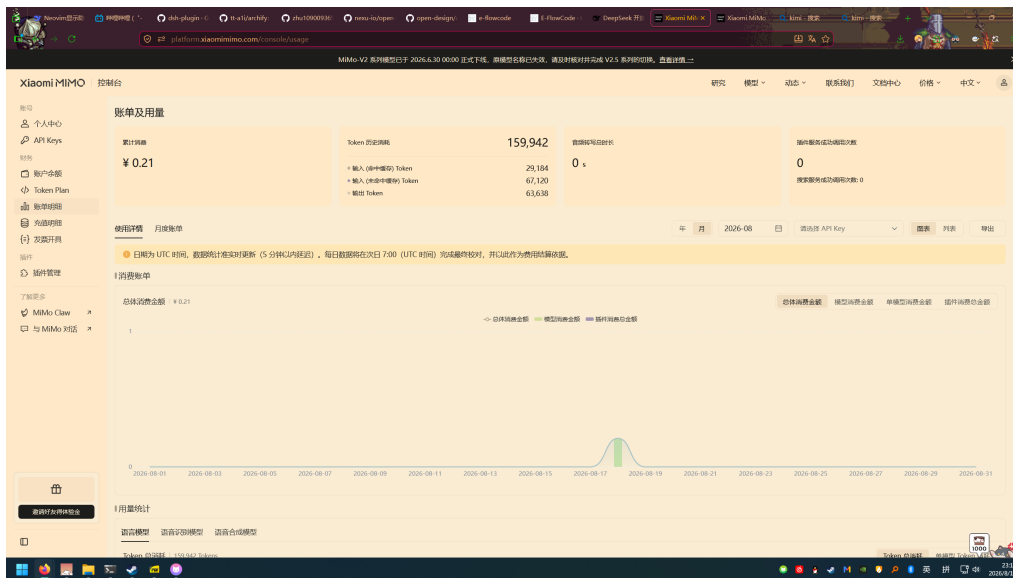


图 2 实验 1: 小米 MiMo 平台 (platform.xiaomimimo.com) 控制台账单及用量页面 (累计消费 ¥0.21, 无 key 明文)。

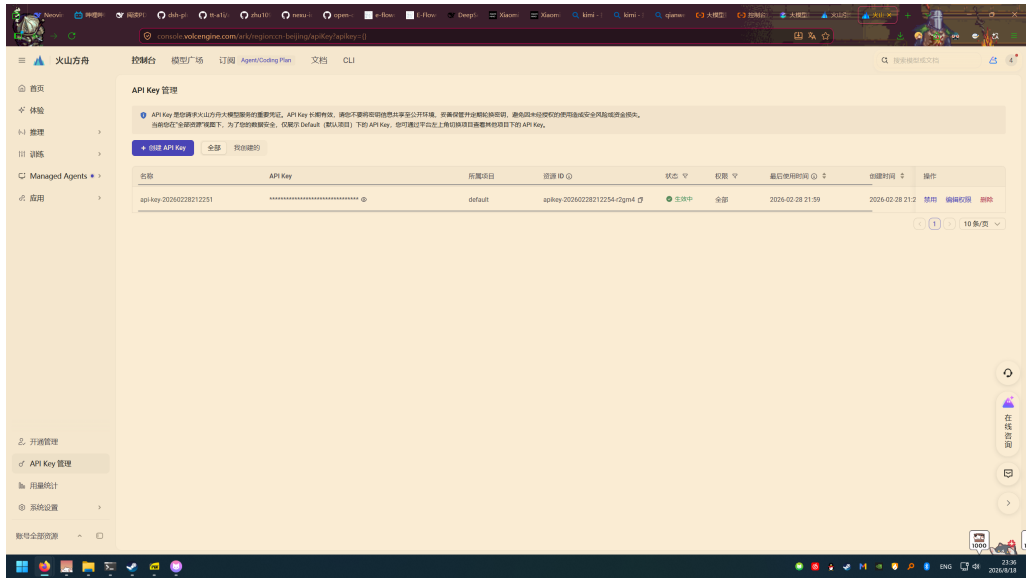


图 3 实验 1: 火山方舟 (console.volcengine.com/ark) API Key 管理页面 (key 以星号遮挡)。

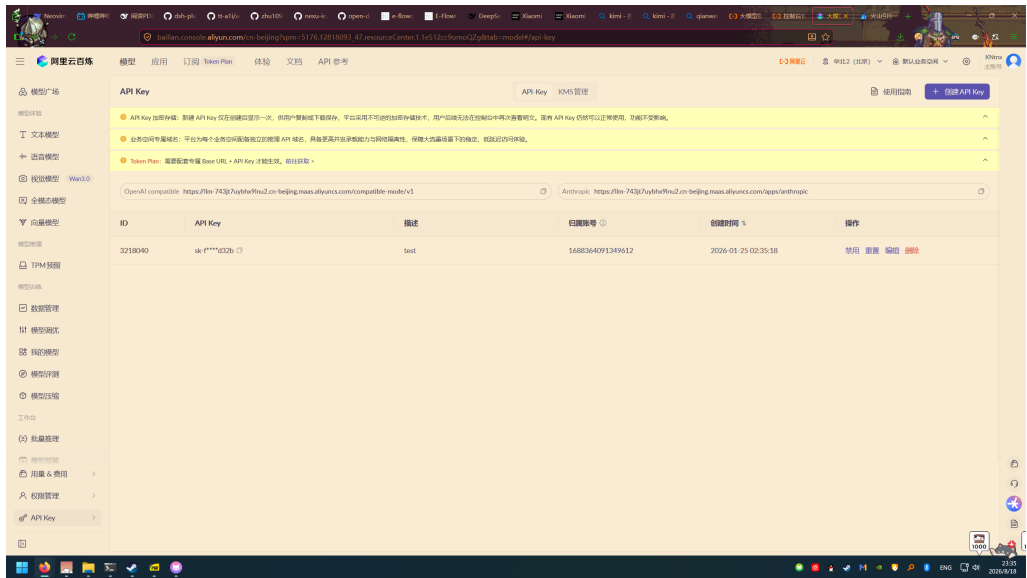


图 4 实验 1: 阿里云百炼 (bailian.console.aliyun.com) API Key 管理页面 (key 以 sk-****d32b 形式部分显示)。

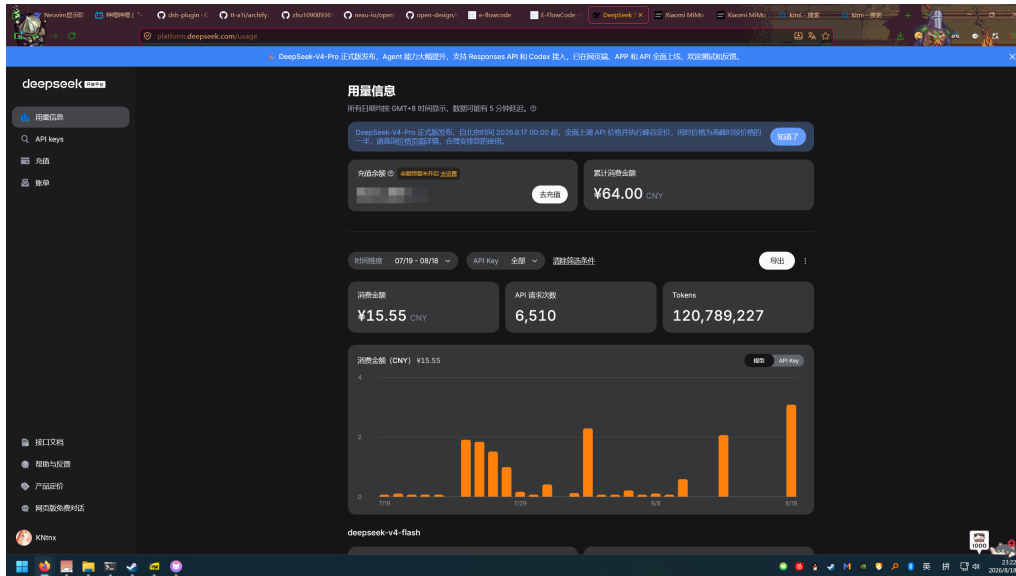


图 5 实验 1: DeepSeek 开放平台 (platform.deepseek.com) 控制台用量信息页面 (充值余额已遮挡; 累计消费为账户历史统计, 无 key 明文)。

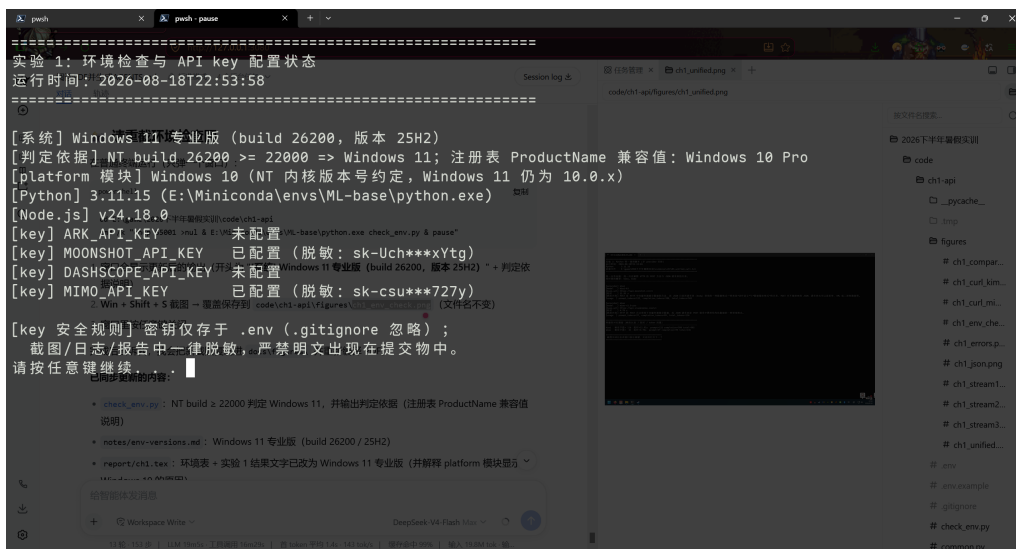


图 6 实验 1: 环境检查与 API key 配置状态的真实运行窗口 (key 已脱敏显示)。

1.4.2 实验 2 结果: curl 调用 (Kimi 与 MiMo)

两个 provider 的 POST /chat/completions 均返回 HTTP 200。首次实验使用 max_tokens=150, 出现了一个值得记录的现象: 由于 kimi-k3 与 mimo-v2.5-pro 都是默认开启思考模式的推理模型, max_tokens 预算被推理 token (reasoning tokens) 大量占用, 导致正文被截断, finish_reason 为 length。以 Kimi 为例 (截断版响应):

```
{"id":"chatcmpl-6a84574c699481545f155cb3",
```

```
"object": "chat.completion", "model": "kimi-k3",
"choices": [{ "index": 0, "message": {
  "role": "assistant", "content": "",      <- 正文为空!
  "reasoning_content": "The user is asking in
    Chinese to introduce ..."},
  "finish_reason": "length" }],
"usage": { "prompt_tokens": 100,
  "completion_tokens": 150,
  "total_tokens": 250,
  "completion_tokens_details": {
    "reasoning_tokens": 147 } } }
```

150 个 completion token 中有 147 个是推理 token，正文完全没有输出。将 max_tokens 提高到 1000 后两个模型都返回完整正文（Kimi：“POST 是 HTTP 中用于向服务器提交数据的请求方法，而 JSON 是 POST 请求体（body）中最常用的数据格式之一...”；MiMo：“POST 方法负责向服务器提交数据，而 JSON 请求体是承载这些数据的常用格式——两者是‘信封’与‘信纸’的关系”）。真实运行窗口见图 7 与图 8，完整响应另见 evidence/ch1/02_curl_kimi_full.txt 与 02_curl_mimo_full.txt。

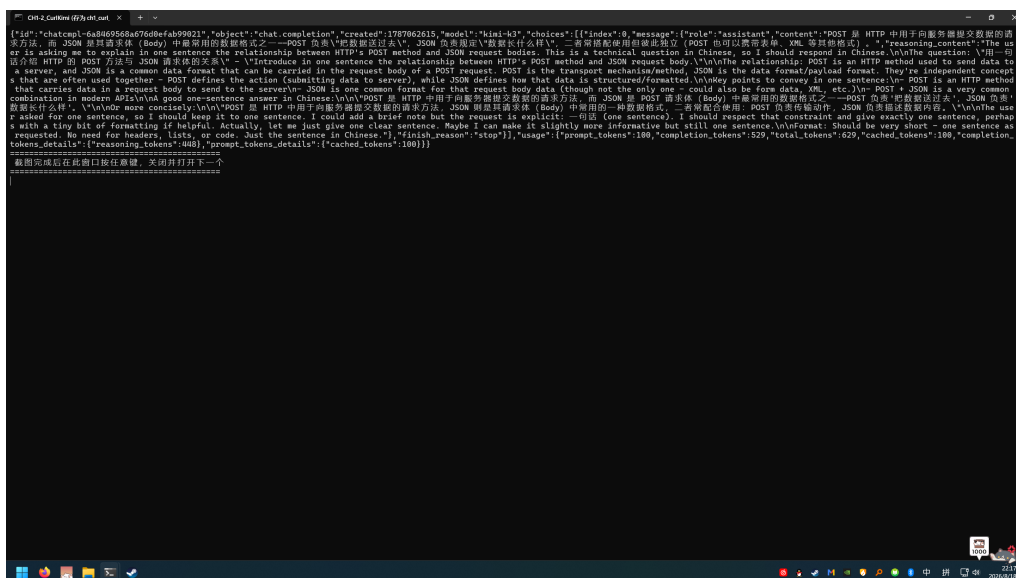


图7 实验2: curl调用Kimi (kimi-k3)的真实运行窗口,返回完整响应与usage。

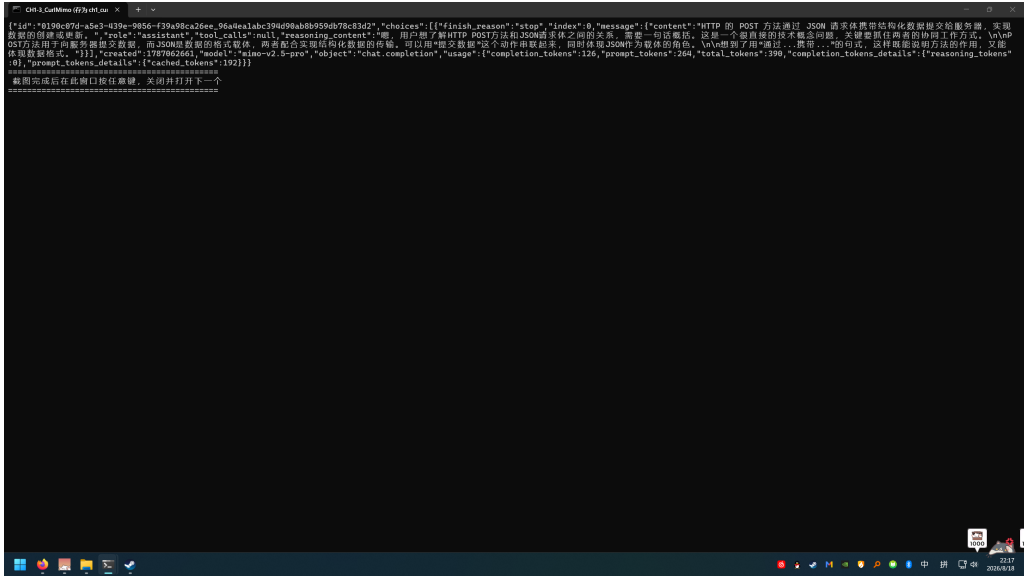


图 8 实验 2: curl 调用小米 MiMo (mimo-v2.5-pro) 的真实运行窗口。

1.4.3 实验 3 结果: Python 统一调用

同一问题(“用一句话解释 HTTP 的 POST 方法与 JSON 请求体的关系”)同时调用 kimi-k3 与 mimo-v2.5-pro, 结果见表 3, 完整输出见 evidence/ch1/03_unified_call.txt。

表 3 统一调用结果对比(同一问题)

指标	kimi-k3	mimo-v2.5-pro
总耗时 (s)	14.12	5.15
响应字数	138	61
prompt_tokens	122	37
completion_tokens	280	130
total_tokens	402	167

两个模型都给出了准确回答。Kimi 的回答更详细(补充了 Content-Type: application/json 的语义), MiMo 的回答更简洁; MiMo 的 prompt_tokens 明显更少, 与其 tokenizer 分词粒度及提示词缓存(cached_tokens)有关。同一份代码只需切换 PROVIDERS 字典即可换平台, 验证了 OpenAI-compatible 的低迁移成本。真实运行窗口见图 9。

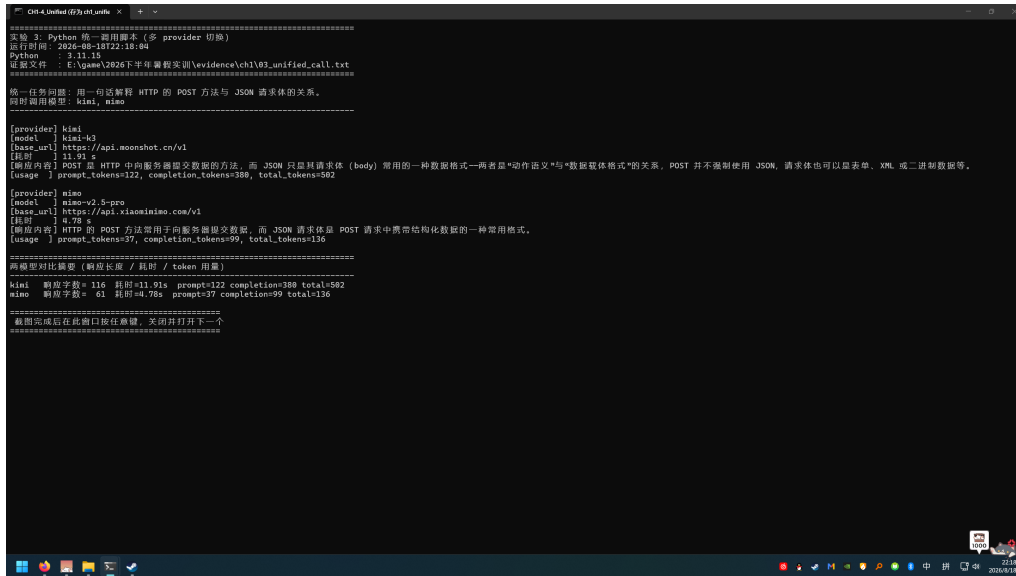


图 9 实验 3：统一调用脚本同时调用 kimi-k3 与 mimo-v2.5-pro 的真实运行窗口，含响应内容、usage 与对比摘要。

1.4.4 实验 4 结果：流式输出

以 kimi-k3 为例，一次流式调用共收到 375 个 chunk。前 160 余个 chunk 的 `delta.content` 为 `None`（模型处于推理阶段），之后才逐字输出正文，最后以 `finish_reason=stop` 结束，并通过空 `choices` 的 chunk 携带 `usage`。关键 chunk 结构（节选）：

```
chunk delta.content=''          <- 首个 chunk (含 role)
chunk delta.content=None       <- 推理阶段：约 160 个 chunk 无正文
... (省略推理 chunk) ...
chunk delta.content='流'       <- 正文开始，逐 token 输出
chunk delta.content='式'
chunk delta.content='输出'
...
chunk finish_reason=stop      <- 结束
[usage chunk] {'completion_tokens': 387,
               'prompt_tokens': 119,
               'completion_tokens_details':
                 {'reasoning_tokens': 294}}
```

流式与普通输出的对比见表 4。流式模式首字节延迟（TTFT）为 2.68 s，之

后内容边生成边到达；普通模式需要等待完整响应（13.24 s）才能开始渲染。两者的最终内容与 token 用量一致。真实运行窗口见图 10（推理阶段 chunk 与正文 chunk）与图 11（流式汇总与普通调用对照）。

对 MiMo 的补充验证（test_mimo_stream_usage.py，证据 05c_mimo_stream_usage.txt）表明：mimo-v2.5-pro 同样支持 stream_options（设为 include_usage=True）参数，流式末尾返回 usage chunk（45 个 chunk，prompt=265、completion=101，含 cached_tokens=192）。两个平台的流式 usage 均可获得，区别在于 Kimi 的 usage 含 reasoning_tokens 明细（推理 token 单独统计），MiMo 的 usage 结构与模型相关。

表 4 流式输出与普通输出对比（kimi-k3）

指标	流式 (stream=True)	普通
首字节延迟 TTFT (s)	2.68	13.24（即总耗时）
总耗时 (s)	14.07	13.24
chunk 数	375	1
usage（prompt/completion/total）	119/387/506	一致

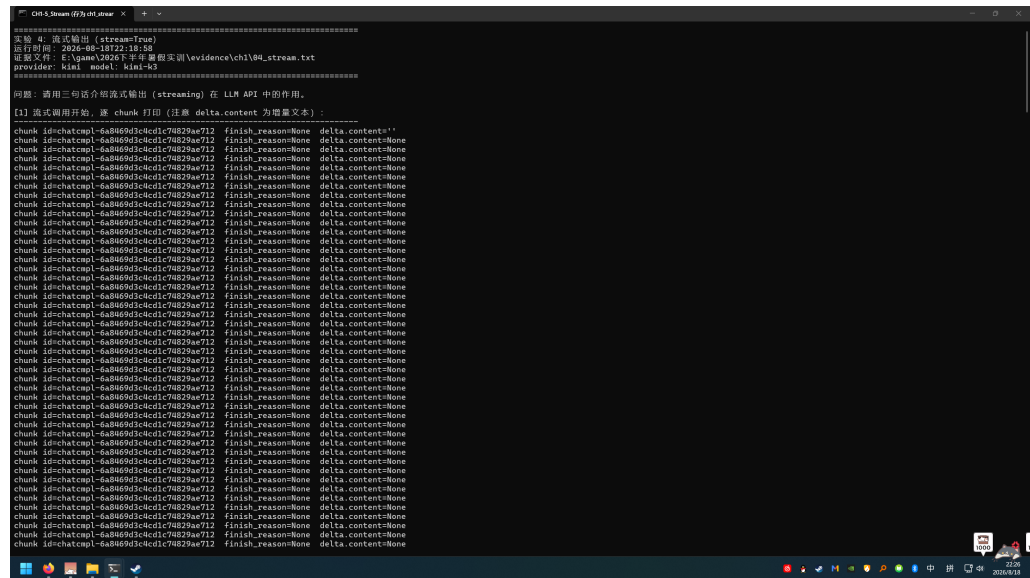


图 10 实验 4：流式输出真实运行窗口——前段为推理阶段 chunk（正文为空），后段为正文逐字 chunk（结构见上文代码块）。

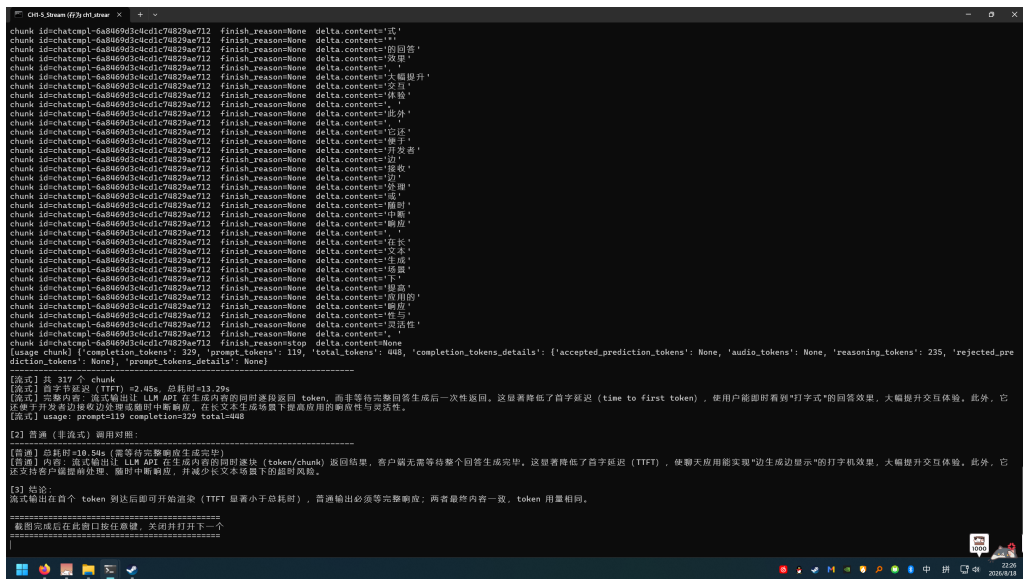


图 11 实验 4: 流式汇总 (chunk 数、TTFT、usage) 与普通调用对照的真实运行窗口。

1.4.5 实验 5 结果: JSON 输出与校验

kimi-k3 在 `response_format={"type":"json_object"}` 下输出合法的课程计划 JSON, `json.loads` 解析成功, 四个字段 (`title`、`tasks`、`deadline`、`risks`) 全部通过结构校验, 且 `tasks` (5 项) 与 `risks` (4 项) 满足数量要求:

```
{
  "title": "学生大模型 API 调用实战教程",
  "tasks": [
    "注册大模型平台账号并获取 API Key, 完成环境变量配置",
    "使用 Python 编写第一个 API 调用脚本, 实现基础文本生成",
    ...],
  "deadline": "2025-06-30",
  "risks": [
    "API 调用超出免费额度产生意外费用",
    "API Key 泄露导致账号被盗用", ...]
}
[json.loads] 解析成功
[结构校验] 通过
```

真实运行窗口见图 12。补充实验 (`test_mimo_json.py`, 证据 `05b_mimo_json_behavior.txt`) 表明: MiMo 在非流式 + `json_object`、非流式 + 纯提示词约

束、流式 + json_object 三种方式下均可输出可解析 JSON，但在实验 7 的对比调用中偶发输出 ``json Markdown 围栏（详见对比分析）。

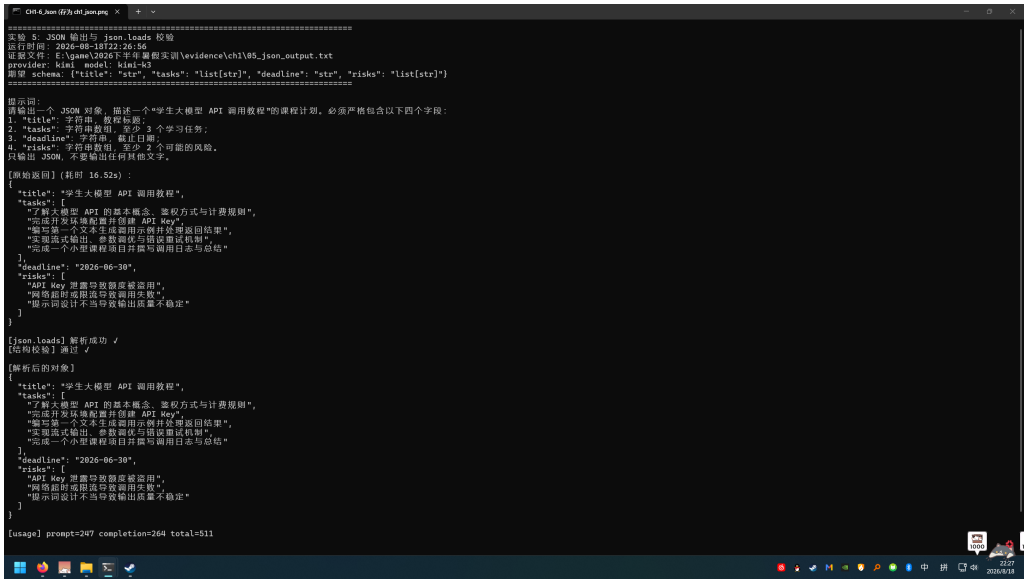


图 12 实验 5: JSON 输出与 json.loads 校验的真实运行窗口，校验与结构检查均通过。

1.4.6 实验 6 结果：错误处理

四类故意触发的错误全部按预期返回，见表 5。Kimi 的错误体遵循 OpenAI 风格（error.message 与 error.type 字段），base_url 错误时返回的是平台自定义格式（code:5、error:url.not_found）。

表 5 错误处理实验结果（kimi-k3）

场景	异常类型	HTTP 状态码	服务端错误信息
错误 API key	AuthenticationError	401	Invalid Authentication
错误模型名	NotFoundError	404	Not found the model ... or Permission denied
错误 base_url	NotFoundError	404	code:5, url.not_found
请求体缺 messages	BadRequestError	400	Invalid request: messages must not be empty

排查要点：401 检查 key 是否有效/权限；400 检查请求体与 model 拼写；404 检查 base_url 路径；429（限流）需要退避重试。真实运行窗口见图 13。

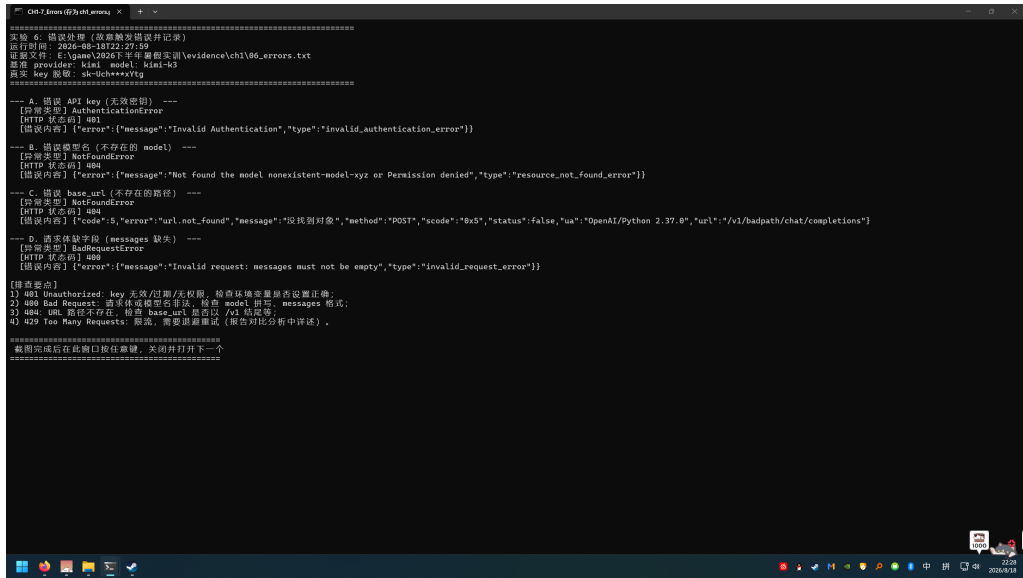


图 13 实验 6：四类错误（401/404/404/400）的真实运行窗口，含异常类型与服务端错误信息。

1.4.7 实验 7 结果：模型对比

同一中文任务（JSON 格式学习计划）流式调用两个模型，结果见表 6 与图 14（延迟柱状图）、图 15（运行窗口对比摘要）。MiMo 的 TTFT 更短（0.90 s），但本次调用总耗时更长且 JSON 输出带 Markdown 围栏导致直接解析失败；Kimi 输出稳定。

表 6 模型对比结果（同一中文任务，流式）

指标	kimi-k3	mimo-v2.5-pro
TTFT (s)	2.57	0.90
总耗时 (s)	12.82	15.80
响应字数	362	405
JSON 直接可解析	是	否（输出 ``json 围栏）
上下文窗口	1,048,576（1M）	1,048,576（1M，最大输出 128K）
输出单价（每 1M tokens）	¥100.00	¥6.00

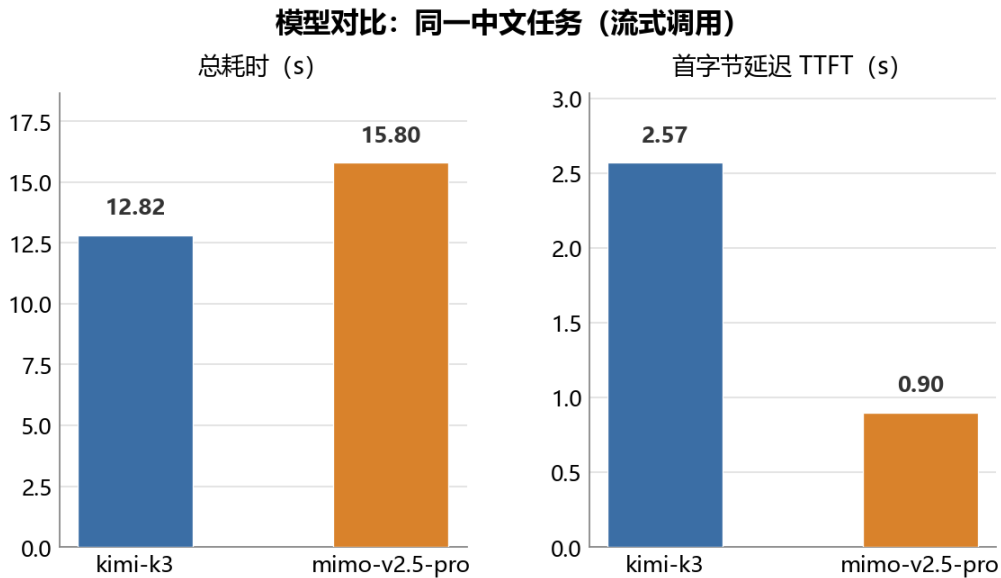


图 14 模型对比：同一中文任务（流式调用）的总耗时与首字节延迟。

```

=====
实验 7: 模型对比 (同一中文任务, 两个模型)
运行时间: 2026-08-18T22:28:25
实验文件: E:\game\2026下半年暑假实训\evideance\ch1\07_compare.txt
=====

给一任务 (要求 JSON 输出):
请用 JSON 输出一个"学生 API 调用入门"的学习计划, 包含字段: ["steps": [3 个步骤字符串], "common_errors": [2 个常见错误字符串], "advice": "一句建议"], 只输出 JSON.

--- 调用 kimi (流式, 测 TTFT) ---
[model] kimi-k3
[TTFT] 2.33s
[总耗时] 14.16s
[响应] {
  "steps": [
    "学习 HTTP 基础知识: 理解 GET/POST 请求方法、请求头、请求体和响应状态码 (如 200, 404, 500) ",
    "使用 Postman 或 curl 工具向一个公开 API (如天气接口 API) 发送第一个请求, 观察返回的 JSON 数据",
    "在代码中实现: 用 Python 的 requests 库调用 API, 解析 JSON 响应并处理可能的异常情况"
  ],
  "common_errors": [
    "忘记在请求头中添加 API Key 或认证信息, 导致返回 401/403 未授权错误",
    "没有检查响应状态码就能解析数据, 遇到错误时程序崩溃或得到意外结果"
  ],
  "advice": "先仔细阅读官方 API 文档, 用最小的请求测试成功后再逐步集成到自己的项目中。"
}
[格式遵循] JSON 可解析 ✓

--- 调用 mimo (流式, 测 TTFT) ---
[model] mimo-v2.5-pro
[TTFT] 1.88s
[总耗时] 14.29s
[响应] {
  "steps": ["理解API的基本概念和作用, 学习HTTP请求方法 (如GET、POST) ", "使用简单API进行实践练习, 如调用公共天气API获取数据", "学习处理API响应和错误, 掌握调试技巧"],
  "common_errors": ["忽略API认证信息 (如API密钥) 导致请求失败", "错误使用HTTP方法或参数格式, 导致数据异常或404错误"],
  "advice": "在开始前仔细阅读API文档, 了解请求和响应格式, 并使用工具 (如Postman) 进行测试。"
}
[格式遵循] JSON 可解析 ✓

=====
对比摘要
=====
| 指标 | kimi | mimo |
|---|---|---|
| 总耗时 (s) | 14.16 | 14.29 |
| TTFT (s) | 2.33 | 1.88 |
| 响应字数 | 300 | 269 |
| JSON 可解析 | True | True |
=====
[图表已生成] E:\game\2026下半年暑假实训\docs\figures\ch1_model_compare.png

[费用说明] token 单价见 notes/env-versions.md (按官方定价记录), 本次实验调用 token 量级为数百, 费用远低于 0.01 元。

=====
截图完成后在此窗口按任意键, 关闭并打开下一个
=====

```

图 15 实验 7: 模型对比的真实运行窗口 (对比摘要: 总耗时、TTFT、响应字数、JSON 可解析)。

1.5 对比分析

- 响应质量:** 两个模型均能准确回答中文技术问题。Kimi 回答更详尽 (会主动补充 Content-Type 等技术细节), MiMo 更简洁。在 JSON 学习计划任务中两者输出内容质量相当 (均覆盖步骤、常见错误与建议三个要素)。
- 格式遵循:** Kimi 在 json_object 模式下输出稳定、无多余内容; MiMo 偶发在 JSON 外加 ``json Markdown 围栏 (补充实验中三种方式均可解

析，说明是采样不稳定而非能力缺失）。工程建议：解析前先剥离代码块围栏，或解析失败时重试一次。

3. **速度**：实验 3 中 MiMo 总耗时（4.78 s）明显快于 Kimi（11.91 s）；实验 7 中两者总耗时接近（12.8 s vs 15.8 s），但 MiMo TTFT（0.90 s）快于 Kimi（2.57 s）。推理模型的延迟受推理长度影响，波动较大，单次测量只能作为量级参考。两个模型均为推理模型，`completion_tokens` 中推理 token 占比高（Kimi 可达 70%+），调用推理模型时 `max_tokens` 必须预留推理空间。
4. **费用**：按官方定价（Kimi K3：输入 ¥20/M、缓存命中 ¥2/M、输出 ¥100/M；MiMo v2.5-pro：输入 ¥3/M、缓存命中 ¥0.025/M、输出 ¥6/M，均以每 1M tokens 计），用各实验日志中的真实 usage 计算（公式：费用 = `prompt` × 输入单价/1e6 + `completion` × 输出单价/1e6，缓存未命中保守口径），汇总见表 7。第 1 章全部 API 调用总费用约 ¥0.169 元（kimi ≈ ¥0.165、mimo ≈ ¥0.004），远低于 0.5 元。需要指出的是：推理模型的推理 token 计入 `completion_tokens` 一并计费（Kimi 的 `reasoning` 占比高，是费用较高的主因）；MiMo 的提示词缓存（`cached_tokens`）可进一步降低重复调用成本。单价与 usage 明细见 `evidence/ch1/08_cost_summary.txt`。

表 7 第 1 章 API 调用费用汇总（官方单价 × 真实 usage，2026-08-18）

实验/调用	prompt	completion	费用（元）
实验 2 curl Kimi（完整）	100	393	0.0413
实验 2 curl Kimi（截断）	100	150	0.0170
实验 2 curl MiMo（完整）	264	84	0.0013
实验 2 curl MiMo（截断）	264	150	0.0017
实验 3 统一调用 kimi	122	380	0.0404
实验 3 统一调用 mimo	37	99	0.0007
实验 4 流式（kimi）	119	329	0.0353
实验 5 JSON（kimi）	247	264	0.0313
kimi-k3 合计			≈ 0.1654
mimo-v2.5-pro 合计			≈ 0.0037
总计			≈ 0.1691

注：费用按缓存未命中保守口径估算（公式见对比分析第 4 条）。其中 3 次调用在日志中检出上下文缓存命中（curl Kimi 完整 cached_tokens=100、curl MiMo 完整 256、curl MiMo 截断 192），若这三行改按缓存单价核算，全章总费用约 ¥0.166（比保守口径低约 ¥0.003）。明细见

evidence/ch1/08_cost_summary.txt。

5. 上下文长度：两模型上下文窗口均为 1M（1,048,576 tokens；MiMo 最大输出 131,072），长文本/整库检索场景均无需分块。差异在于：Kimi 始终推理且输出单价高，长上下文大输出场景成本显著高于 MiMo；MiMo 输出上限（128K）低于 Kimi 的完整 1M 输出空间（Kimi 输出上限以官方文档为准），超长生成任务需注意。
6. 限流与错误恢复：Kimi 的错误体为标准 OpenAI 格式，易于程序化解析；MiMo 未实测 429（本章未触发限流）。两个平台均返回 HTTP 状态码，配合 OpenAI SDK 的异常类型（AuthenticationError / NotFoundError / BadRequestError）可直接定位问题。
7. 文档易用性：Kimi（platform.moonshot.cn）与小米 MiMo（mimo.mi.com）均提供 OpenAI-compatible 快速开始文档；MiMo 文档明确标注思考模式下 temperature/top_p 无效，Kimi 需要从文档中确认 kimi-k3 的推理特性。整体上两者的接入成本相近，均只需 base_url + key + model 三项配

置。

1.6 可复现教程

本章实验的全部脚本位于 `code/ch1-api/`，复现步骤为：

1. 复制 `.env.example` 为 `.env`，填入至少两个平台的 API key；
2. 使用 Miniconda ML-base 环境 (Python 3.11.15)，按 `requirements.txt` 安装依赖 (`pip install -r requirements.txt`)；
3. 依次运行：`unified_call.py kimi mimo`、`stream_call.py kimi`、`json_output.py kimi`、`error_cases.py kimi`、`compare_models.py kimi mimo`；
4. 每个脚本自动把真实输出写入 `evidence/ch1/` (UTF-8 日志)。

参考文献

- [1] Moonshot AI. Kimi API 快速开始 [EB/OL]. <https://platform.moonshot.cn/docs/guide/start-using-kimi-api>，访问日期 2026-08-17.
- [2] 阿里云. 阿里云百炼首次调用千问 API[EB/OL]. <https://help.aliyun.com/zh/model-studio/getting-started/first-api-call-to-qwen>，访问日期 2026-08-17.
- [3] 火山引擎. 火山方舟快速入门 [EB/OL]. <https://www.volcengine.com/docs/82379/1298454>，访问日期 2026-08-17.
- [4] 小米. Xiaomi MiMo API 开放平台——首次调用 [EB/OL]. <https://mimo.mi.com/docs/zh-CN/quick-start/summary/first-api-call>，访问日期 2026-08-18.
- [5] MDN. Overview of HTTP[EB/OL]. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>，访问日期 2026-08-17.
- [6] Bray T. The JavaScript Object Notation (JSON) Data Interchange Format: RFC 8259[S]. IETF, 2017. <https://datatracker.ietf.org/doc/html/rfc8259>，访问日期 2026-08-17.

- [7] curl. curl official site[EB/OL]. <https://curl.se/>, 访问日期 2026-08-17.
- [8] OpenAI. OpenAI Python API library[EB/OL]. <https://github.com/openai/openai-python>, 访问日期 2026-08-17.

2 第二章：提示词到智能体的演化

2.1 背景

prompt engineering 解决的是“如何问”的问题，适合单次任务和短链路任务。随着任务变长，模型需要看到项目文件、约束、历史、工具说明、测试结果和外部数据，于是问题变成“模型应该看到什么”，这就是 context engineering。再往后，模型不仅要看，还要行动：读文件、写文件、跑命令、搜索、调用 API、开子任务，这需要 harness engineering。最后，多步任务必须不断观察结果、修正计划、判断是否结束，这才进入 loop agent。

演化的内在逻辑：任务复杂度越高、时间跨度越长、外部环境越真实、错误代价越大，工程重心就越从“写好一句 prompt”转向“管理上下文、约束工具、记录状态、验证结果和设计循环”。本章用一个真实任务（写一份“学生 API 调用入门教程”）做四层递进实验，量化每一层带来的变化。

2.2 概念背景：四层各控制什么

表 8 四个 engineering 层级：工程对象与解决的问题

层级	工程对象	主要解决问题	控制面
Prompt	一条/一组指令、示例、输出格式约束	让单次模型调用更稳定、怎么问更符合格式	
Context	模型本次采样能看到的全部 token	在有限上下文内放入“刚好需要”的信息	看什么
Harness	模型外部的运行时：工具、权限、沙箱、日志、检查器	让模型能安全感知环境、能做什么行动、被约束、被验证	
Loop agent	“观察 - 思考 - 调工具 - 观察 - 判断是否继续”的控制循环	处理多步、长时、不确定、何时停需反馈修正的任务	

四个层级不是替代关系，而是外层包住内层（图 16）：loop 的每一轮都需要 harness 提供工具与约束，harness 的每一次模型请求都需要 context 提供材料，context 的组织方式又由 prompt 的结构决定。

2.2.1 为什么 context 不是把资料全部塞进 prompt

上下文包不采用“全量投射”而是“精选摘要 + 引用”，原因有四：

1. **有限上下文窗口**：token 预算有限，全量放入必然挤压任务相关信息的空间；
2. **信息污染**：无关材料会让模型跑偏（工程语境称 **context rot**，上下文腐化），文档里与任务无关的章节、广告话术都是噪音；
3. **成本与延迟**：token 越多费用与耗时越高（第 1 章费用记录：费用 = token × 单价，上下文占用量直接计价）；
4. **可维护性**：资料有版本与来源，全量塞入无法逐项标注更新策略；正确做法是把“刚好需要”的信息按来源摘要进上下文，完整文档放在文件系统/检索里按需取用——这正是本章上下文包的设计（官方文档摘要 + 学生画像 + 模板要求，每项标注来源）。

2.2.2 为什么工具说明也是 prompt/context 的一部分

模型只有“看到”工具说明（名字、参数、用途、输出约定与使用时机），才能决定是否调用、如何填参数。工具说明随注册进入系统提示词组装（第 4 章验证：注册工具 schema 自动流入系统提示词——“schemas flow into the system-prompt assembly automatically”），因此工具说明本质上是 context engineering 的对象：参数描述清晰、用途边界明确的说明，模型才用得对；工具说明含糊（参数含义不明、无输出约定）会导致模型反复试错、填错参数甚至干脆不用工具。



图 16 prompt → context → harness → loop agent 四层嵌套演化示意。

2.2.3 harness 与 framework、IDE、MCP、plugin、sandbox、CI 的关系

harness 不是某一个具体技术，而是“模型外部的运行时”这一角色的总称，它与常见概念的关系是：

- **framework**（框架）：harness 可以用框架实现（如 LangChain/ Pi/DSH 各自的内核），但框架是软件形态，harness 是职责——同一个 harness 职责可以由不同框架承载；
- **IDE**（**Agent IDE** / 编辑器）：IDE 是 harness 能力的交互载体——DSH Web UI、CodeBuddy 等把 harness 的工具、会话、审计与权限包装成聊天/编辑器界面；IDE 与 harness 是“界面 vs 运行时”的关系，同一天 harness 可以驱动不同前端；
- **MCP**：MCP 是工具接入的开放协议（模型-工具之间的标准化传输层），harness 是执行环境——harness 通过 MCP 客户端接入外部 MCP server 的工具，MCP 解决“工具怎么连”，harness 解决“工具怎么被安全地调度与验证”；
- **plugin**：插件是 harness 的组合单元——DSH 的“Everything is a plugin”表明模型、工具、沙箱、UI 都可作为插件挂载进 harness 运行时；
- **sandbox**：沙箱是 harness 的权限边界之一——工具执行的隔离环境（文件系统/网络/进程限制），属于 harness engineering 的安全约束面；
- **CI**：CI 是 harness 质量保障的延伸——把检查器、回归测试、评审流程接入 agent 循环（如第 2 章 loop 层的 rubric 检查器，放大到仓库级就是 CI 化的 agent 流水线）。

2.3 实验设计：同一任务四层递进

1. **Prompt 层**：只给模型一句任务：“写一个面向零基础学生的大模型 API 调用入门教程”，无任何额外信息；
2. **Context 层**：在系统提示中加入上下文包：Kimi 官方文档摘要（申请/调用/计费要点）、OpenAI-compatible 迁移说明、目标学生基础、报告模板要求；
3. **Harness 层**：保留上下文包，并给模型一套**真实工具集**：`write_file`（写入沙箱文件）、`run_tests`（运行独立测试脚本）、`bash`（执行命令，黑名

单保护)——即“脚本 + 测试 + JSON 校验 + rubrics”的完整能力(循环模式改编自 Claude Code 教学实现 s01/s02: TOOLS schema + tool_result 回传)。模型必须实际调用工具才能完成任务,且只允许写入沙箱目录;

4. **Loop 层:** 同一套工具集,但把检查器接入循环——每轮若测试未全部通过,程序把失败项注入下一轮消息要求修改,最多 4 轮,直到测试全过(16/16)或达上限。

四层使用同一任务、同一模型(mimo-v2.5, 非 pro 档; 输入 ¥1/M、输出 ¥2/M, 见官方定价页)、同一上下文包(后三层),只改变“约束与循环”的层级,量化各层增量收益。选 MiMo 而非第 1 章的 kimi-k3 出于成本考虑: 四层多轮调用(第 3/4 层为多回合工具循环),kimi 需 ¥1 以上、MiMo v2.5 约 ¥0.05——对比实验多轮调用用便宜模型性价比更高(平台差异已在第 1 章对比)。实验脚本见 code/ch2-evolution/(layers.py、harness_tools.py、rubric.py、tests/test_tutorial.py),原始日志见 evidence/ch2/02_layers.txt (2026-08-30 完整运行,含每回合工具调用记录、测试报告与 [END] 完成标记)。

2.4 实验结果

四层实验均使用 mimo-v2.5,任务与上下文包相同,只改变约束层级。检查结果、耗时/usage 统计见表 9;完整原始输出(含模型编造的端点、错误模型名与引用日期等细节,见各层分析)见 evidence/ch2/02_layers.txt。

表 9 四层递进实验结果（同一任务，mimo-v2.5，2026-08-30 运行）

指标	1 层 Prompt	2 层 Context	3 层 Harness	4 层 Loop
rubric 覆盖	8/8	8/8	16/16*	16/16*
输出格式	自由 Mark-down	自由 Mark-down	JSON 文件	JSON 文件
工具调用	—	—	是（实际调用）	是（实际调用）
聚焦平台	否（泛化 OpenAI）	是	是	是
单次耗时 (s)	40.1	45.4	多回合工具	多回合工具
usage 输入/输出	265/2509	326/2681	见 * 注释	见 * 注释
循环轮数	—	—	2 回合上限	第 1 轮 16/16

数据说明：本次实验为 2026-08-30 完整运行（日志含开始时间与 [END] 结束标记）。* 第 3/4 层的测试为 `tests/test_tutorial.py` 的 16 项检查（JSON 五字段 + rubric 八项 + 引用格式）；第 3/4 层为多回合工具循环（每次模型请求都真实计费），逐回合 token 明细已在 `harness_tools.py` 增补（下次运行记录），本次以第 1/2 层实测 usage 核算并结合模型回合数估算：四层合计费用约 ¥0.05，以平台控制台账单为准。各层的日志原文证据见相应小节（“日志证明”）。

2.4.1 第 1 层：只有一句话

只给模型一句任务（“写一个面向零基础学生的大模型 API 调用入门教程”）。rubric 关键词检查显示 8/8 全部命中，但细读输出发现三个问题：

1. **任务平台错位：**模型把示例完全建立在 **OpenAI** 上（`gpt-3.5-turbo`、`api.openai.com`，还顺带提了百度文心一言），而实训实际使用国内平台（第 1 章 Kimi/MiMo）——prompt 层没有上下文约束时，模型按自己的先验选择平台；
2. **环境与接口信息失真：**‘示例代码’用的 `openai.ChatCompletion.create`（API 已废弃的旧写法）、成本提示“每 1000 token 约 0.002 美元”（不是国产平台计费），文末“本教程最后更新：2024 年 7 月”也是编造的日期——rubric‘示例代码’项被关键词命中，但内容不可直接复现；

3. **引用缺失**：文末没有给出任何官方文档链接（rubric ‘参考链接’ 项未命中关键词，靠“推荐学习资源”泛泛表述凑过）。

结论：关键词覆盖率高是“泛化的覆盖”——输出与当前任务环境（实训平台、学生基础、报告模板）不匹配，甚至换用另一套技术栈（OpenAI）；这正是 context 层要解决的问题。

日志证明（evidence/ch2/02_layers.txt 本节原文节选，括号内为措辞提炼，其余逐字保留）：

[Prompt] 40.1s | usage: 265/2509

[输出]

大模型 API 调用入门教程

****——面向零基础学生的友好指南****

```
response = openai.ChatCompletion.create(
```

```
    model="gpt-3.5-turbo",
```

```
    url = "https://api.openai.com/..."
```

```
    ... (文末) 本教程最后更新：2024 年 7 月
```

[rubric 检查] 未覆盖要点：全部覆盖

[要点覆盖] 8/8

2.4.2 第 2 层：加入上下文包

在系统提示中加入上下文包（Kimi 官方文档摘要、OpenAI-compatible 迁移说明、目标学生基础、报告模板要求）。输出立刻发生三个可见变化：

1. **平台对齐**：全篇以 Kimi 为例，给出 `api.moonshot.cn/v1/chat/completions` 的真实端点与 `sk-` 开头的 `key` 说明，与上下文包中的官方文档摘要一致；
2. **学生基础对齐**：“以 Kimi 平台为例，面向零基础学生”，环境准备表写明“Python 3 已具备（课程要求）”——上下文包中的学生画像直接进入输出；
3. **结构对齐**：出现“背景说明/环境准备/实验步骤/结果/对比分析”的报告结构雏形，费用说明指向官方定价页。

但请同时注意第 2 层输出里的两处事实性错误：**(1) 模型名错误**——示例代码与结果中使用的模型是 `moonshot-v1-8k`，而上下文包（第 1 章实测与官方摘要）给

出的是 kimi-k3; (2) 引用日期编造——参考资料“访问日期：2024 年 7 月”，上下文包没有提供任何访问日期。也就是说：context 层解决了“看什么/举例贴不贴近”，但无法保证事实性细节正确（模型名、日期、版本号等易被模型填充为似是而非的值）——这正是第 3 层引入 外部校验与测试 的动机。

日志证明（evidence/ch2/02_layers.txt 本节原文节选）：

```
[Context] 45.4s | usage: 326/2681
[输出]
# 大模型 API 调用入门教程
## 一、背景说明
...（以 Kimi 为例，API 端点 api.moonshot.cn/v1/...）
    payload = {
        "model": "moonshot-v1-8k",
    ...
*   访问日期：2024年7月
[rubric 检查] 未覆盖要点：全部覆盖
[要点覆盖] 8/8
```

2.4.3 第 3 层：Harness（工具 + 脚本 + 测试）

本层把“约束”从提示词彻底挪到模型外部：不依赖模型自觉遵守格式，而是给模型一套必须实际使用的工具（改编自 Claude Code 教学实现 s01_agent_loop.py / s02_tool_use.py 的工具循环模式——TOOLS schema 分发 + tool_result 回传）：

```
工具 1  write_file(path, content)    写入沙箱文件
工具 2  run_tests(path)              运行 16 项测试
工具 3  bash(command)               执行命令（黑名单）
循环    tool_call -> tool_result -> 下一轮
```

本层实测（max_tool_turns=2：只允许“写 + 测”各一次，验证面完整即强制结束，不自动迭代；以下为 evidence/ch2/02_layers.txt 日志原文）：

```
[turn 1] 模型调用工具 write_file(...)
-> 结果：Wrote 3378 chars -> sandbox
```


[turn 2] 模型调用工具 `run_tests(...)`

-> 结果: PASS 全 16 项 (节选)

[turn 3] 工具回合已达上限 2, 强制结束

(harness 验证面完成, 不自动迭代)

== 测试汇总: 16/16 项通过 ==

与第 2 层的可见差异: 模型不再“交出文本等人工检查”, 而是把教程落盘成文件、运行独立测试、逐项看到 PASS/FAIL——判定结论由 `test_tutorial.py` 给出 (16 项: 五字段结构 + rubric 八项 + 引用 URL/日期格式), 模型不能自评, 绕不过去 (不写文件、不跑测试就没有完成信号)。安全边界同时生效: 写入限于沙箱目录、`bash` 带黑名单 (改编 s01 的危险命令拦截)。

本层边界 (必须指出): (1) 测试脚本判定的是“结构与关键词”——16/16 通过不代表内容真实 (引用日期真伪需要人工/外部核对, 测试只做了格式检查并输出“日期标注”提示); (2) `max_tool_turns=2` 意味着本层不自动迭代——测试通过了 (16/16) 最好; 若未通过, harness 记录失败就结束 (修正动作留给第 4 层)。

2.4.4 第 4 层: Loop agent

同一套真实工具集, 但把检查器接入循环: 每轮运行后程序解析测试报告, 若未全部通过, 把失败项注入下一轮消息 (“上一轮测试未通过以下检查项: ...”), 最多 4 轮, 直到 16/16 或达上限。

本层实测: 第 1 轮即通过 (以下为 `evidence/ch2/02_layers.txt` 日志原文):

--- 第 1 轮 ---

[turn 1] 模型调用工具 `write_file(...)`

-> 结果: Wrote 6997 chars -> sandbox

[turn 2] 模型调用工具 `run_tests(...)`

-> 结果: == 测试汇总: 16/16 项通过 ==

[完成] 第 1 轮通过全部 16 项测试!

[最终输出] 已通过验证 (8 个模块:

背景/环境/步骤/结果/对比/安全/错误处理/引用)

第 1 轮输出覆盖背景说明/环境准备/实验步骤/结果分析/对比分析/安全注意/错误处理/参考资料 8 个模块 (模型在最终答复中给出模块清单), 测试逐项判定

通过。这个结果说明两件事：

1. 当 prompt/context/harness 充分时，loop 的修正轮次可以很少（本实验第 1 轮通过）——循环的价值不在于“必须多轮”，而在于“有检查器才能可靠地知道何时停止”；
2. 停止条件的天花板 = 测试的边界：测试脚本覆盖结构/关键词/ 格式，“16/16 通过”不等于“内容真实”（引用日期真伪本是测试里标注的“待人工核对”项）——不可验证的事实性内容需要外部验证手段（如脚本比对官方页面修订时间），否则 Agent 会在错误事实上“自信地”收敛。这也是反思节“哪些任务用 agent 风险更大”的实证。

2.5 Agent 的本质

定义：Agent 是一个由模型驱动、在外部 harness 约束下运行的闭环系统。它维护任务状态，把合适上下文交给模型，允许模型选择工具或动作，把观察结果写回状态，并根据目标、检查器或人工反馈决定继续、修正、求助或停止。单次 LLM API 调用不是 agent；只有模型没有工具和状态也不是完整 agent；只有 while loop 但没有目标检查、权限和可观测性也不可靠。

本实验使用的循环机制抽象为最小 agent loop 伪代码（与第 4 层实验 layers.py 的实现对应）：

```
state = { task, messages=[],
          上下文包, 检查器=rubric, 上限=4 }
while 检查器未全部通过 and 轮数 < 上限:
    view      = 组上下文(state)      # context 组装
    reply     = model.complete(view) # 模型请求
    if reply 含工具调用:
        # harness: 校验 -> 执行 -> 写回
        results += execute(reply.tools)
        # 本实验无工具分支,
        # 工具循环实证见第 3/4 章
    messages += 模型回复 + 工具结果    # 状态更新
    report    = 检查器判定(state)    # 停止条件
```

```
if report 未通过:
    # 修正输入
    messages += 未过项反馈
print(通过/失败, 轮数, 耗时, token)
```

第4层实验真实运行的是**检查器反馈环**：模型生成 → `json.loads + rubric` 判定 → 未通过项并入下一轮消息 → 再生成，直到通过或到上限。它证明的是 **agent** 的“环”与“停止条件”侧面。完整的**工具调用循环**（模型请求 → 工具调用 → 工具结果 → 下一轮请求）由第3章 **Pi** 源码分析与第4章 **DSH** 插件实测（`greet` 工具被模型调用、结果写回会话）承接——两个侧面合起来才构成定义中的完整 **agent**。

2.6 反思

1. **哪些任务不需要 agent**：单轮问答、格式固定的转换任务，一次调用即可，`loop` 只是浪费延迟与 `token`。
2. **哪些任务用 agent 风险更大**：高副作用（支付、删除）任务，必须配合权限与人工确认；不可验证的任务（主观写作），检查器无法提供可靠的停止信号。
3. **如何设计停止条件**：优先“检查器全通过”；其次轮数/预算上限；再次人工确认。停止条件决定了 **agent** 的可靠性边界。

参考文献

- [1] Moonshot AI. Kimi API 快速开始 [EB/OL]. <https://platform.moonshot.cn/docs/guide/start-using-kimi-api>, 访问日期 2026-08-18.
- [2] Anthropic. Prompt engineering overview[EB/OL]. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>, 访问日期 2026-08-17.
- [3] Anthropic. Effective context engineering for AI agents[EB/OL]. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>, 访问日期 2026-08-17.

- [4] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]. ICLR 2023. <https://arxiv.org/abs/2210.03629>, 访问日期 2026-08-17.
- [5] Anthropic. Building Effective AI Agents[EB/OL]. <https://www.anthropic.com/research/building-effective-ai-agents>, 访问日期 2026-08-17.
- [6] OpenAI. Harness engineering[EB/OL]. <https://openai.com/index/harness-engineering/>, 访问日期 2026-08-17.

3 第三章：Pi-core 源码与工具调用分析

3.1 背景

源码分析的目标不是通读每一行，而是抓住 `agent harness` 的关键链路：模型怎样收到工具 `schema`、怎样输出 `tool call`、运行时怎样校验和执行、工具结果怎样进入会话历史、循环怎样决定继续或停止。本章选择 `Pi` 作为分析对象——它是较适合教学的 `agent harness` 例子：仓库目录为 `packages/agent`，包名为 `@earendil-works/pi-agent-core(v0.84.2, commit 59a71b2)`，其 `agent-loop.ts` 可以直接看到工具调用的执行路径。

仓库：<https://github.com/earendil-works/pi>

commit: 59a71b2 (2026-08-18 克隆)

包: `@earendil-works/pi-agent-core 0.84.2`

3.2 关键文件与职责

表 10 Pi 工具调用链路关键文件 (≥8 个)

文件路径	在工具调用链路中的职责
<code>packages/agent/package.json</code>	包名 @earendil-works/pi-agent-core、依赖与定位
<code>packages/agent/src/types.ts</code>	AgentLoopConfig、AgentToolCall、AgentToolResult、before/after hook、executionMode
<code>packages/agent/src/agent-loop.ts</code>	主循环：流式响应、提取 toolCall、串/并行执行、toolResult 入上下文
<code>packages/ai/src/types.ts</code>	底层 Message 等消息类型
<code>packages/ai/src/api/*</code>	不同 provider 把统一消息转换为 OpenAI/Anthropic/Google/Mistral 协议
<code>packages/coding-agent/src/core/tools/*</code>	内置 read/write/edit/bash/grep/find/ls 工具定义与执行
<code>.../tools/tool-definition-wrapper.ts</code>	ToolDefinition 与 AgentTool 的包装关系
<code>.../core/extensions/wrapper.ts</code>	扩展工具包装进 AgentTool，工具名反馈给运行时
<code>.../core/agent-session.ts</code>	工具注册表、启用、系统提示重建、tool event 映射到会话 UI

3.3 工具调用链路与时序

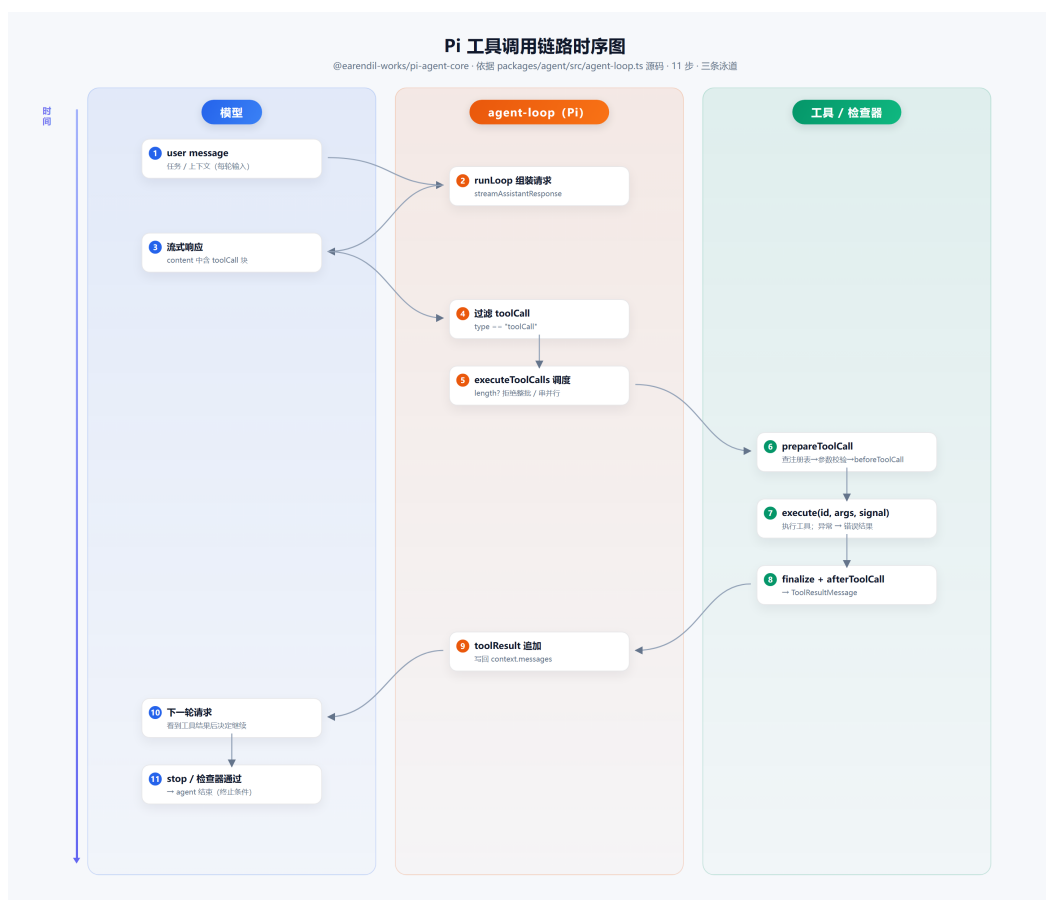


图 17 Pi 工具调用链路时序图：模型请求 → 工具调用 → 工具结果 → 下一轮请求（依据 agent-loop.ts 源码绘制）。

3.3.1 主链路（源码确认，agent-loop.ts）

user message

-> runAgentLoop / runLoop (155 行)

-> streamAssistantResponse (304 行)

-> assistant message 的 content 中过滤 type == "toolCall"

-> stopReason == "length" ?

是: failToolCallsFromTruncatedMessage (390 行) [截断保护]

否: executeToolCalls (420 行)

检测 executionMode == "sequential" (工具可自声明)

串行 executeToolCallsSequential (442 行)

或并行 executeToolCallsParallel (498 行)

-> 每个 toolCall:

```

prepareToolCall (609 行)
    工具不存在 -> "Tool xxx not found" 错误结果
prepareArguments (工具可预处理参数)
validateToolArguments (参数校验)
beforeToolCall hook: 可 block (reason + terminate)
executePreparedToolCall (679 行)
    执行 tool.execute(id, args, signal, partialUpdate)
    异常 -> createErrorToolResult
finalizeExecutedToolCall (722 行)
    afterToolCall hook 可替换结果
-> createToolResultMessage (786 行)
    -> append 到 currentContext.messages
-> 下一轮 LLM 请求看到 tool result
-> stopReason == "stop" / shouldStopAfterTurn 时退出

```

3.3.2 关键函数解释 (≥3 个)

1. **runLoop(agent-loop.ts:155):** 双层循环。外层持续轮询 steering 消息（用户排队输入）；内层在“还有工具调用或待处理消息”时持续：流式请求模型 → 提取 toolCall → 执行工具批 → 结果追加进 currentContext.messages → 再请求模型，直到没有新的工具调用或 stopReason 为 stop/error/aborted。工具结果不经过任何中间存储，直接写入上下文供下一轮请求使用。
2. **prepareToolCall(agent-loop.ts:609):** 工具执行的前置关卡。按名字查注册表（未知工具立即返回错误结果）；调用 prepareArguments（工具可规范化参数）；调用 validateToolArguments 做 schema 校验；再执行 config.beforeToolCall hook——hook 返回 block 时可以拒绝执行并附带 reason，terminate 时整批终止。任何异常都被捕获转为错误结果（不中断循环）。
3. **executeToolCalls(agent-loop.ts:420):** 调度器。若配置要求串行、或任一被调用工具声明 executionMode === "sequential"，则整批串行；否则并行执行。每个工具的执行通过 executePreparedToolCall 完成——携带 AbortSignal 支持取消，并支持通过 partialResult 回调流式

上报执行进度。

4. **AgentLoopConfig(types.ts:149)**: 循环配置中枢。`convertToLlm`(AgentMessage→LLM 消息, 必填)、`transformContext`(上下文裁剪/外部注入钩子)、`getApiKey`(动态取 key, 支持短时 OAuth token)、`shouldStopAfterTurn`(优雅停止条件)。

3.4 错误路径 (≥4 条, 均有源码依据)

1. 未知工具: `prepareToolCall` 中注册表找不到工具 → “Tool xxx not found” 错误结果, `isError: true` (618–622 行)。
2. 参数校验失败: `validateToolArguments` 抛错 → 被 `catch` 转为 `createErrorToolResult` (627、670–675 行)。
3. 工具执行抛异常: `executePreparedToolCall` 的 `catch` 分支把异常信息包装为错误结果 (710–715 行)。
4. 输出 **token 截断**: `stopReason` 为 “length” 时拒绝执行全部工具调用, 提示 “arguments may be truncated, re-issue” (390–415 行, 函数 `failToolCalls-FromTruncatedMessage`) ——宁可报错重来, 也不执行可能残缺的参数。
5. 取消/中止: `signal.aborted` 时返回 “Operation aborted”; 串行循环中中止会中断剩余调用 (638–663、487–489 行)。
6. 策略拦截: `beforeToolCall` 返回 `block: true` 时拒绝执行, `terminate` 级联终止整批 (645–655 行)。

3.5 设计评价 (≥2 个)

1. **截断保护设计**: 对 “length” 停止的消息不执行任何工具调用。这是容易被忽略的健壮性细节——模型输出被 token 上限截断时, 工具参数可能不完整, 执行会浪费配额甚至产生错误副作用; Pi 选择把整批工具调用标记为错误并要求模型重发。
2. **hook 分层 (before/afterToolCall)**: 执行前拦截 (权限/策略/审计) 与执行后替换 (结果加工) 分离, 且 `block` 支持 `terminate` 级联——权限拒绝可以干净地终止整个工具批, 而不是让后续工具继续执行。

3. 串行/并行自动选择: 工具可自声明 `executionMode: "sequential"` (如编辑类工具需要顺序), 运行时按“任一工具要求串行则整批串行”的规则调度, 兼顾正确性与吞吐。
4. 上下文钩子分离: `transformContext` (上下文窗口裁剪、外部注入) 与 `convertToLlm` (消息协议转换) 职责分开, 上下文管理不需要关心 `provider` 差异。

3.6 与 DSH 的横向比较

本章同时对照了 DSH (DeepSeek Harness, 见第 4 章) 的实现 (`packages/core/tools/src/index.ts`、`agent-loop/src/agent.ts` 与 `tool-calls.ts`, 基于官方文档与本地安装源码)。

表 11 Pi 与 DSH 工具调用链路对比

维度	Pi (pi-agent-core)	DSH
插件模型	<code>extensions</code> 扩展 + 工具注册表	Everything is a plugin (Cordis 插件行)
工具注册	<code>AgentTool</code> (<code>execute</code> 手写, <code>schema</code> 由 <code>typebox</code> 定义)	<code>ctx.tools.register</code> (<code>defineTool</code> 编译 <code>schema</code> 并自动校验参数)
执行管道	<code>before/afterToolCall</code> hook	<code>tools/pre-execute</code> 、 <code>tools/execute</code> 、 <code>tools/post-execute</code> 、 <code>tools/result</code> 事件管道
配置方式	<code>AgentLoopConfig</code> (代码内对象)	<code>cordis.yml</code> patch 分层 (<code>profile/bundle/patch</code>)
上下文钩子	<code>transformContext</code> / <code>convertToLlm</code>	上下文组装进系统提示词, 会话日志持久化
截断保护	<code>stopReason=length</code> 拒绝整批工具	(第 4 章插件开发中由 <code>exec.signal</code> 与 <code>schema</code> 校验覆盖)

两种实现都遵循“模型输出工具调用 → 校验 → 执行 → 结果回写上下文 → 下一轮”的统一范式。差异在于工程组织: Pi 把调度逻辑集中在 `agent-loop.ts` 内联函数, hook 少而清晰; DSH 把管道拆成可插拔事件 (`pre/execute/post/result`),

并借助 Cordis 让工具、策略、UI 都可独立挂载——这对应了从单机工具链到插件化运行时的架构演化（与第 2 章 harness engineering 的概念呼应）。

参考文献

- [1] earendil-works. Pi GitHub 仓库 (monorepo) [EB/OL]. <https://github.com/earendil-works/pi>, 访问日期 2026-08-18.
- [2] earendil-works. pi-agent-core package.json[EB/OL]. <https://github.com/earendil-works/pi/blob/main/packages/agent/package.json>, 访问日期 2026-08-18.
- [3] earendil-works. packages/agent/src/agent-loop.ts[EB/OL]. <https://github.com/earendil-works/pi/blob/main/packages/agent/src/agent-loop.ts>, 访问日期 2026-08-18.
- [4] earendil-works. packages/agent/src/types.ts[EB/OL]. <https://github.com/earendil-works/pi/blob/main/packages/agent/src/types.ts>, 访问日期 2026-08-18.
- [5] DeepSeek. DSH Architecture 与 Tool execution pipeline[EB/OL]. <https://github.com/deepseek-ai/DeepSeek-Harness/blob/master/docs/architecture.md>, 访问日期 2026-08-18.

4 第四章：DeepSeek Harness 融入具体应用

4.1 背景

DeepSeek Harness (dsh) 是 DeepSeek 官方开源的 agent harness，目前处于 developer preview 阶段（本章实际使用两个版本形态：早期以 `npx @deepseek-ai/dsh@0.1.0-rc.6` 验证（解析安装为 0.1.0-rc.7），后续以源码编译方式运行官方发布版本 0.1.2-alpha.1（commit `cd5ef81`），两个形态的差异见 4.2 版本对照）。它的核心设计是“Everything is a plugin”：模型、工具、skills、sessions、sandboxes、storage、loops、scheduling 和 UI 都由插件组合。它基于 Cordis 运行时，插件通过 `apply(ctx)` 向上下文注册能力，通过 `profile`、`bundle` 和 `patch` 组合出不同运行时。

本章的目标是把 DSH 放进真实研发流程：在本机 WSL2 上以源码编译、从源码启动的方式搭建 DSH 运行环境（与任务书建议的“`git clone + pnpm install + pnpm dsh web`”流程一致），研读官方插件教程并实际完成（`scratch-plugin + cordis.yml overlay + 终端输出`；注册 `greet` 工具并在 Web UI 会话中被模型真实调用），再对照官方 `minimal shape` 开发业务工具插件（`dev_environment_snapshot` 巡检工具与 LaTeX 报告脚手架生成器），最后验证远程访问安全方案。

4.2 环境

本章以 **WSL2 源码编译方式**为主环境（与任务书建议一致：远端 Linux + `git clone + pnpm` 构建），同时保留早期 `npx` 方式的验证记录，两套形态对照见表 12。

表 12 第 4 章实验环境（源码方式为主 + npx 方式对照）

项目	版本/配置
操作系统	Windows 11 专业版 (build 26200 / 25H2); WSL2 Ubuntu 24.04 LTS
Node.js	v24.18.0 (Windows) / v22.22.3 (WSL; 满足 ^22.19.0 >=24.0.0)
pnpm	11.22.0 (WSL, 源码方式使用)
DSH 源码 (主)	git clone github.com/deepseek-ai/DeepSeek-Harness
运行版本	release 0.1.2-alpha.1 (commit cd5ef81)
启动方式	pnpm dsh web --patch ./scratch-plugin/cordis.yml
DSH (npx 方式, 对照)	@deepseek-ai/dsh@0.1.0-rc.6 (解析安装 0.1.0-rc.7, 组件 0.1.0-rc.8)
@deepseek-ai/dsh-tools	0.1.0-rc.6 (npx 形态) / 源码仓库同源
@deepseek-ai/cordis	4.0.1
模型	DeepSeek-V4 Flash (Web UI Settings → Models 配置; 凭据走 DSH credentials 服务, 仓库与报告无明文 key)
远端子目录	~/game/agent-lab/ (projects、secrets.env、deepseek-harness)
运行日期	2026-08-18 (npx 早期验证); 2026-08-30 (源码方式完成官方教程)

源码编译 (本机 WSL2, 主环境, 与任务书"源码方式"一致):

```
git clone github.com/deepseek-ai/DeepSeek-Harness.git
cd deepseek-harness && pnpm install && pnpm dsh web
```

本章实际运行的官方教程命令:

```
pnpm dsh web --patch ./scratch-plugin/cordis.yml
```

npx 早期验证 (对照形态):

```
dsh --version # -> rc.6 (npx 实际安装 rc.7)
```

```
dsh --help # --profile / --patch / --dump-config
```

4.3 官方教程复现

本章以 DSH 官方开发文档为主教程（GitHub `deepseek-ai/DeepSeek-Harness` 的 `docs/user/develop/basic/` 目录，访问日期 2026-08-18），按 “Your first plugin” 与 “Build a tool” 的顺序研读并在 **WSL2 源码编译环境中实际完成**：插件源码与 `cordis.yml` 见 `code/ch4-dsh/follow-official/scratch-plugin/`（与 WSL 中 `/game/agent-lab/deepseek-harness/scratch-plugin/` 一致），运行命令与终端输出见 4.3.1，Web UI 会话中的工具调用见 4.3.2。任务书必做实验完成情况对照见表 13。

表 13 第 4 章任务书必做实验 1-5 完成情况对照

必做实验	状态	证据
1. 跟做 Your first plugin（终端输出）	已完成	4.3.1 节；图 18（终端截图）
2. 跟做 Build a tool（greet 返回 Hello, Ada!）	已完成	4.3.2 节；图 19（Web UI 会话截图）
3. greet 改造为业务工具（dev_environment_snapshot）	已完成	4.3.3 节；图 20
4. 解释 inject 必要性与 define-Tool 各字段	已完成	原理分析节（14）
5. 至少一个安全约束	已完成	4.3.3 节安全边界

4.3.1 Your first plugin：创建插件并看到终端输出

1. 在源码仓库根目录创建 `scratch-plugin/src/my-plugin.ts` —— 插件 = 一个导出 `name`、`inject`、`apply` 的模块（`apply(ctx)` 在挂载时执行）：

```
import type { Context } from '@deepseek-ai/cordis'
import { defineTool } from '@deepseek-ai/dsh-tools'
```

```

export const name = 'my-first-plugin'
export const inject = ['tools']    // 依赖 tools 服务

export function apply(ctx: Context) {
  // 实验 1: 打印日志
  console.log('[my-plugin] 插件已加载! ')
  // 实验 2: 注册 greet 工具 (见 4.3.2)
  ctx.tools.register(defineTool({ ... }))
}

```

(为可读性省略正文；完整源码见 `code/ch4-dsh/follow-official/scratch-plugin/src/my-plugin.ts`。注：终端实际输出含对勾表情（源码 `console.log` 中为“my-plugin 插件已加载！”加对勾），因等宽字体不含该字形，本代码块以文字代替——真实输出见图 18。）

1. 创建 `scratch-plugin/cordis.yml` Web overlay，把本地插件插入插件树（官方要求 `name` 为绝对路径）：

```

- insert:
  - id: my-plugin
    name: .../game/agent-lab/deepseek-harness/
      scratch-plugin/src/my-plugin.ts

```

(`name` 为插件源码绝对路径，如上以省略号缩写；完整路径见仓库 `code/ch4-dsh/follow-official/scratch-plugin/cordis.yml`。)

1. 用官方命令启动 Web UI 并加载 overlay（源码方式，仓库根目录执行）：

```
pnpm dsh web --patch ./scratch-plugin/cordis.yml
```

终端输出（真实运行，图 18）：启动时 `apply` 被挂载执行，终端打印 `[my-plugin] 插件已加载!`，随后服务器就绪——与官方教程预期“The terminal prints `[hello-plugin] plugin loaded! during startup`”一致（官方示例输出为英文 `[hello-plugin] plugin loaded!`，本章中文输出为同一行为）：

```

$ node --import tsx esm/apps/cli/src/bin.ts \
  web --patch ./scratch-plugin/cordis.yml

```

[my-plugin] 插件已加载! <- apply() 终端输出
dsh web: http://127.0.0.1:3080/?token=... (服务就绪)

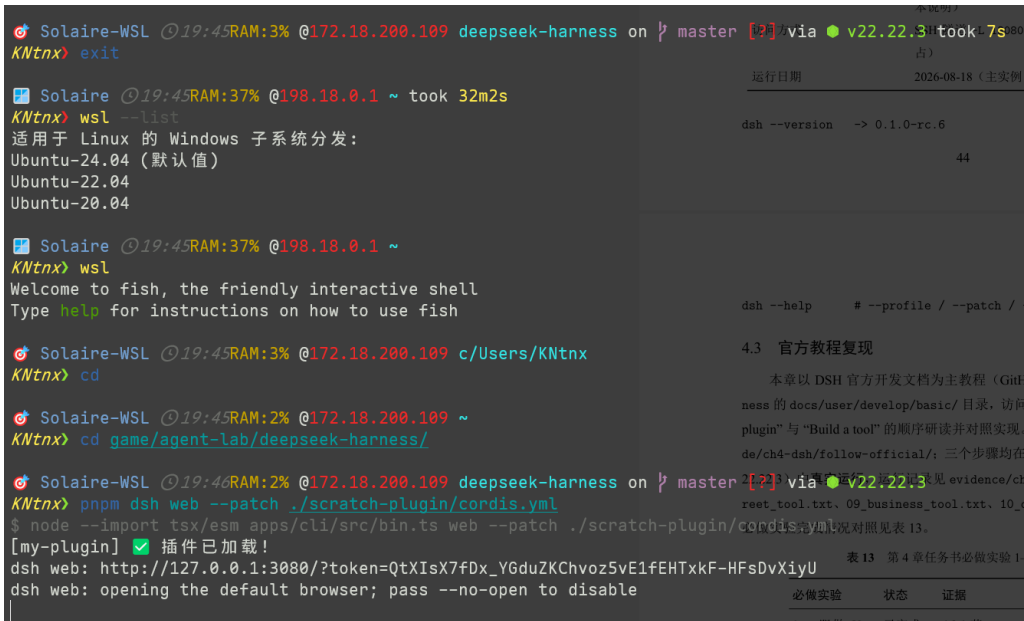


图 18 跟做 Your first plugin: pnpm dsh web --patch ./scratch-plugin/cordis.yml 的真实终端输出 [my-plugin] 插件已加载! (WSL2 源码编译环境, Node 22.22.3)。

4.3.2 Build a tool: greet 被模型真实调用

在上面的 my-plugin.ts 中同时注册官方教程的工具 greet (defineTool 声明参数 name、输出 schema 与 render, execute 返回 Hello, \${args.name}!)。随后在 DSH Web UI (模型: DeepSeek-V4 Flash; 工作区: agent-lab) 新建会话并发送任务: “Use the greet tool to greet Ada.”

运行结果 (真实会话, 图 19): 模型自动发起 1 次工具调用——greet, 输入 {"name": "Ada"}, 工具输出 Hello, Ada!, 模型据此回答 “I greeted Ada with a friendly 'Hello, Ada!' ”。会话统计: 1 秒 2 步、输入 164 token / 输出 57 token、缓存命中 49.5%。

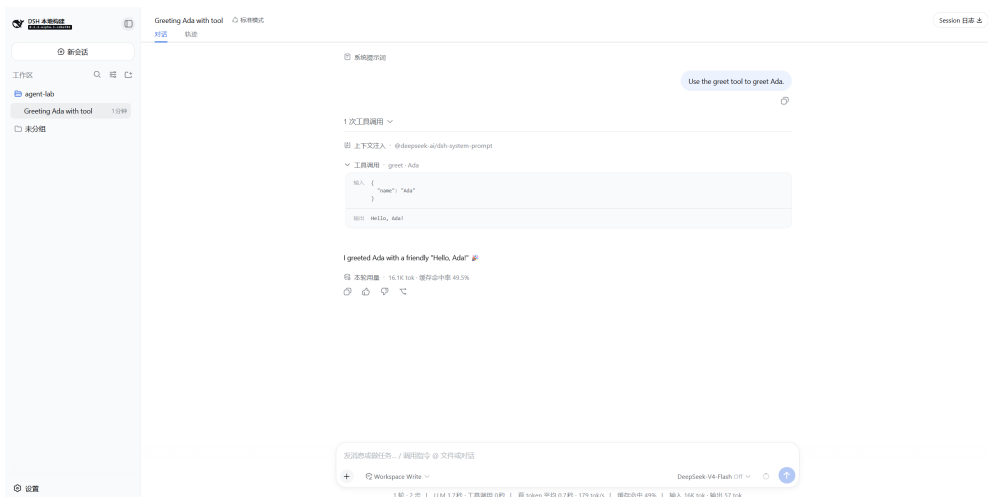


图 19 跟做 Build a tool: DSH Web UI 会话中模型真实调用 greet 工具（输入 name=Ada → 输出 Hello, Ada!），模型回答“I greeted Ada with a friendly 'Hello, Ada!’”。

这就是“模型请求 → 工具调用 → 工具结果 → 下一轮请求”的一次真实闭环：greet 的 execute 返回 Hello, Ada!（官方实现逐字对应），经 render 后作为工具结果写回会话，模型把结果转成自然语言。（inject = ['tools'] 与 defineTool 各字段的职责详见原理分析节（表 14）。）

4.3.3 业务工具：dev_environment_snapshot（greet 的改造）

任务书必做实验 3 要求把官方 greet 改造成业务工具（示例：“返回当前目录、git 分支、Node/Python 版本、package manager、关键文件是否存在”）——在 scratch-plugin/src/ 下新增插件 dev-snapshot.ts（与 my-plugin.ts 同目录，cordis.yml 以第二条 insert 挂载），完整实现见 code/ch4-dsh/follow-official/scratch-plugin/src/dev-snapshot.ts。核心实现（execute 的探测逻辑）：

```
const cwd = process.cwd()
gitBranch    = execSync('git branch --show-current') // 非 git 目录
nodeVersion  = process.version                       // 如 v22.22.3
pythonVersion = execSync('python3 --version')
packageManager = 检测 pnpm / npm / yarn 锁文件
keyFiles     = { package.json, README.md, .gitignore }
```

Web UI 会话实测（与 4.3.2 同一会话：“使用 dev-snapshot 快照输出”，图 20）：模型调用 dev_environment_snapshot，工具返回并展开快照——

工作目录：/home/KNtnx/game/agent-lab/deepseek-harness

Git 分支: master Node 版本: v22.22.3
Python 版本: Python 3.12.3 包管理器: pnpm
关键文件: package.json 存在 README 存在 .gitignore 存在

与任务书示例逐项对应（当前目录/Git 分支/Node/Python 版本/包管理器/ 关键文件是否存在）。模型据此回答“环境已就绪，可创建软件依赖...”。会话统计：工具调用 1 次、输入 166 token/输出 230 token、缓存命中 49.5%。

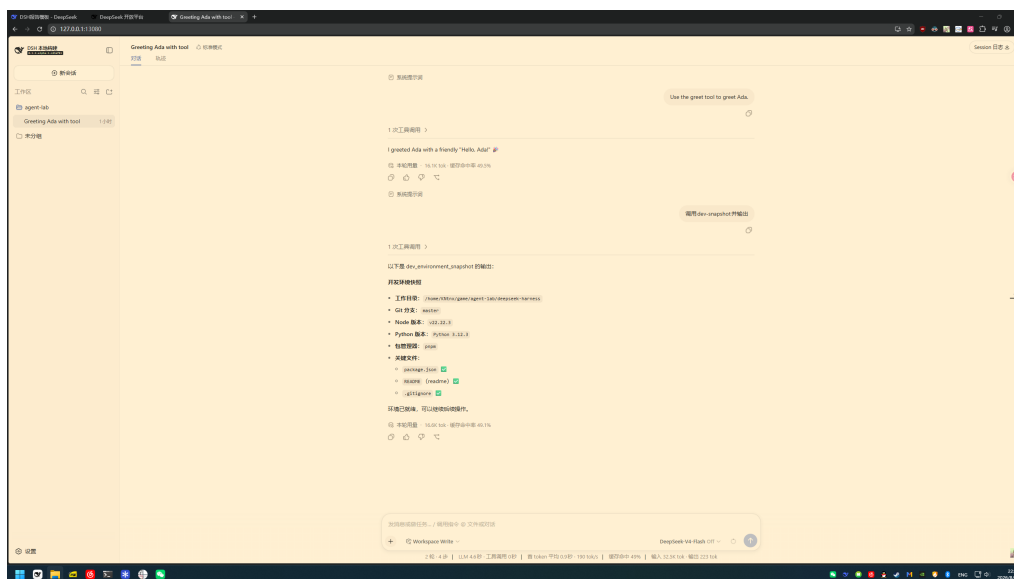


图 20 Web UI 会话调用 dev_environment_snapshot: 模型发起 1 次工具调用，返回工作目录/Git 分支（master）/Node v22.22.3/Python 3.12.3/ pnpm/关键文件三连存在，模型据此回复“环境已就绪”。

说明：该工具与第 5 章 devops-scout 插件的巡检能力同源（第 5 章为其强化版：增加依赖解析/测试探测/变更安全审计）。

4.3.4 安全边界（任务书必做实验 5 的设计与分析）

按任务书第 5 条要求设计安全约束；本章两个插件(greet、dev_environment_snapshot)的约束落实情况如下：

1. **只读优先：**dev_environment_snapshot 只读取目录/版本信息与文件存在性，不写任何文件；
2. **写入范围受限**（若扩展为生成类工具）：只向参数指定的输出目录（相对工作区）写入生成的文件，不触碰其他路径；
3. **敏感文件不读：**工具不读取.env、密钥文件（只报存在性，见 4.3.3 节 dev_environment_snapshot）；

4. **key 不进工具返回**：工具返回内容只含文件清单与检查结果，不涉及任何凭据；
5. **命令白名单（默认拒绝）**：`dev_environment_snapshot` 的 `parameters` 为空——模型无法传入任何命令或路径参数，`execute` 内部只执行硬编码的只读探测命令（`git branch --show-current`、`python3 --version`、文件存在性检查），等效于最简白的名单（一切外部输入默认拒绝）；
6. **高危操作人工确认**：本插件无高危操作；若后续扩展为执行命令的工具，将接入 DSH 的 `tools/pre-execute` 权限管道（`allow/deny/ask` 策略）。

4.4 原理分析

4.4.1 `inject = ['tools']` 为什么必要

`tools` 是 DSH 宿主提供的工具注册表服务。插件在 `apply` 里使用 `ctx.tools` 前必须先声明 `inject = ['tools']`：Cordis 只有对声明了硬依赖的插件才会等待对应服务就绪后再挂载；未声明时 `ctx.tools` 可能为 `undefined`，或直接被 Guard 拒绝（“service tools is not declared”）。`inject` 同时把插件挂载顺序显式化，避免竞态。

本章两个插件的印证：`my-plugin.ts`（4.3.1，插件名 `my-first-plugin`）与 `dev-snapshot.ts`（4.3.3，插件名 `dev-snapshot-plugin`）均在文件头部声明 `export const inject = ['tools']` 后，才在各自的 `apply(ctx)` 中调用 `ctx.tools.register(defineTool(...))`——`greet` 由 `my-first-plugin` 注册（4.3.2），`dev_environment_snapshot` 由 `dev-snapshot-plugin` 注册（4.3.3），两者经 `cordis.yml` 的两条 `insert` 分别挂载。注意：即便 `dev_environment_snapshot` 的 `parameters` 为空（无参数工具，见 4.3.4 的“默认拒绝”安全边界），也要声明 `inject=['tools']`——`inject` 声明的是宿主能力依赖（工具注册表服务），与工具参数多少无关。省略的后果：`ctx.tools` 未就绪，插件加载时被 Guard 拒绝或运行时抛出“service tools is not declared”。

4.4.2 defineTool 各字段职责

表 14 defineTool 关键字段职责（对齐官方 `cookbook/adding-a-tool`；“本章用法”列为 4.3 节两个真实工具）

字段	职责	本章用法
<code>name</code>	工具唯一名（模型调用与注册表索引）	<code>greet</code> （4.3.2）、 <code>dev_environment_snapshot</code> （4.3.3）
<code>parameters</code>	参数 <code>schema</code> ，编译为 JSON Schema， <code>execute</code> 前自动校验	<code>greet</code> : <code>name</code> 必填； <code>dev_snapshot</code> : <code>{}</code> （无参）
<code>output.schema</code>	规范化返回值的 <code>schema</code> （程序可消费）	<code>greet</code> : <code>{type:'string'}</code> ；快照对象（ <code>cwd/gitBranch/...additionalProperties:false</code> ）
<code>output.render</code>	把规范化值转成模型可见文本块	<code>greet</code> 返回 <code>text</code> 块；快照 <code>JSON.stringify(v, null, 2)</code> 展开
<code>execute</code>	实际执行； <code>args</code> 已校验、需尊重 <code>exec.signal</code>	<code>greet</code> : <code>return Hello, \${args.name}!</code> ；快照：只读探测（见 4.3.3）

（表中字段均可与 4.3.2/4.3.3 的源码片段与图 19、图 20 直接对应；`output.render` 的“展开式”效果即图 20 中的可读快照。）

4.4.3 工具进入模型上下文的链路

注册的工具 `schema` 会自动进入系统提示词组装（“`schemas flow into the system-prompt assembly automatically`”）；模型按 `schema` 生成工具调用参数，注册表校验后进入执行管道（`pre-execute` 策略 / `execute` / `post-execute`），规范化结果经 `render` 后作为工具结果写入会话——与第 3 章分析的“模型请求—工具调用—工具结果—下一轮请求”链路一致，只是实现方从 Pi 换成了 DSH。

4.4.4 profile / bundle / patch 分层

DSH 配置按层叠加：bundle（npm 包，声明 dsh.bundle.patch）→ profile 自身 patch → home patch → 命令行 --patch。本章 18 使用的即命令行 --patch（overlay）路径；bundle 路径上 dsh plugin add 检测到 dsh.bundle 声明会把插件追加进 profile 的 bundles 列表，--dump-config 可查看合成后的插件层；home patch 与命令行 patch 不能重复挂载同一插件 id，否则报 duplicate loader entry id（配置排错的一个真实案例）。

4.4.5 远程运行与安全访问（WSL 视作独立远程主机）

任务书的远程方案（远端 Linux + SSH 隧道访问 3080）在本机 WSL2 上再次完整验证。此处把 **WSL 当作一台独立的远程 Linux 主机**：访问不使用 WSL 的 localhost 镜像/转发机制（该机制视为不存在），连接路径为“WSL 虚拟网卡 eth0 地址 → sshd:22”。**主机信息**：主机名 Solaire-WSL、用户 KNtnx、sshd 监听 0.0.0.0:22（仅本机可达）、专用密钥 id_dsh_wsl（无口令，仅用于本隧道；authorized_keys 权限 600、.ssh 目录 700）。网卡地址为动态分配（WSL 重启会变），可用以下命令查询：

```
wsl -d Ubuntu-24.04 -- ip addr show eth0 | grep inet
```

```
# -> inet <eth0-ip>/20 ...（动态，每次启动可能变化）
```

```
# 远端（WSL，即"远程主机"）启动 DSH（仅监听 loopback，不暴露网卡）：
```

```
export DSH_HOME=~/.dsh-agent-lab
```

```
npx -y @deepseek-ai/dsh@0.1.0-rc.6 web
```

```
# -> dsh web: http://127.0.0.1:3080
```

```
# 本机（Windows）建立 SSH 端口转发（标准远程主机形态）：
```

```
ssh -N -L 13080:127.0.0.1:3080 \
```

```
KNtnx@<wsl-ip> -p 22 -i ~/.ssh/id_dsh_wsl
```

```
# <wsl-ip> 为上面的 eth0 地址（实践时若本机 3080 已被本地 DSH 实例
```

```
# 占用，隧道映射到 13080 即可；浏览器打开 http://127.0.0.1:13080）
```

端到端验证（2026-08-30，均为真实终端与真实会话）：

1. 远端启动(图 21):WSL 中 `pnpm dsh web --patch ./scratch-plugin/cordis.yml` 启动 (源码方式), 终端打印 `[my-plugin]` 插件已加载! 与 `dsh web`: `http://127.0.0.1:3080/?token=...`;
2. 本机建立转发(图 22):`ssh -N -L 13080:127.0.0.1:3080 KNtnx@<wsl-ip> -p 22 -i ~/.ssh/id_dsh_wsl` (连接成功进入保持态);
3. 验证 (图 23): 本机 `curl http://127.0.0.1:13080/` 返回 `dsh web authentication required; reopen the URL printed by dsh web.` (DSH 令牌鉴权拦截——请求已完整穿透隧道到达 DSH 服务, 若隧道不通将是连接错误而非 HTTP 响应); 再访问带 token 的 URL (`?token=...`, 图内已打码) 即正常打开 Web UI (图 24)。

```

Solaire-WSL @22:38AM:2% @172.18.200.189 deepseek-harness on ? master [?] via @ v22.22.3 took 1h28m19s
~/ttnx
LICENSE      docs/      pnpm-workspace.yaml  tsconfig.client.tsbuildinfo  vitest.shared.ts
BENCHMARK.md README.l10n.yaml  knip.json            pytest.ini                  tsconfig.host.json          vitest.snapshot.config.ts
BRAND_GUIDELINES.l10n.yaml  README.md  lefthook.yml         pnpm/                        scratch-plugin/             tsconfig.host.tsbuildinfo  vitest.web-stress.config.ts
BRAND_GUIDELINES.zh.md     README.zh.md  native/              scripts/                    snapshot/                   tsconfig.json              vitest.web.config.ts
CLIQUE.md                  SAFETY.l10n.yaml  SAFETY.md            package.json                snapshots/                  vendor/                    vitest.web.perf.config.ts
CONTRIBUTING.l10n.yaml    SAFETY.zh.md  THIRDPARTY_NOTICES.md  patches/                   tsconfig.base.client.json   vitest.config.ts           website/
CONTRIBUTING.md           THIRDPARTY_NOTICES.md  epss/                pnpm-lock.yaml              tsconfig.client.json        vitest.config.ts           vitest.expected.config.ts
CONTRIBUTING.zh.md
Solaire-WSL @22:38AM:2% @172.18.200.189 deepseek-harness on ? master [?] via @ v22.22.3
~/ttnx
K/ttnx> cd scratch-plugin/
Solaire-WSL @22:38AM:2% @172.18.200.189 deepseek-harness/scratch-plugin on ? master [?]
~/ttnx
K/ttnx> cd ..
Solaire-WSL @22:38AM:3% @172.18.200.189 deepseek-harness on ? master [?] via @ v22.22.3
~/ttnx
K/ttnx> cd ..
Solaire-WSL @22:38AM:3% @172.18.200.189 ~/game/agent-lab
~/ttnx
K/ttnx> npx
Solaire-WSL @22:38AM:3% @172.18.200.189 ~/game/agent-lab took 1h28m17s
~/ttnx
K/ttnx> cd deepseek-harness/
Solaire-WSL @22:38AM:3% @172.18.200.189 deepseek-harness on ? master [?] via @ v22.22.3
~/ttnx
K/ttnx> pnpm dsh web --patch ./scratch-plugin/cordis.yml
[my-plugin] 插件已加载!
dsh web: http://127.0.0.1:3080/?token=[token已打码]
dsh web: opening the default browser; pass --no-open to disable
  
```

图 21 SSH 验证第 1 步: 远端 (WSL 独立主机) 启动 DSH——`pnpm dsh web --patch ./scratch-plugin/cordis.yml`, 终端输出 `[my-plugin]` 插件已加载! 与 `dsh web`: `http://127.0.0.1:3080/?token=...` (token 已打码)。

```

Solaire @22:39AM:37% @198.18.0.1 ~
K/ttnx> ssh -N -L 13080:127.0.0.1:3080 KNtnx@172.18.200.189 -p 22 -i ~/.ssh/id_dsh_wsl
  
```

图 22 SSH 验证第 2 步: 本机建立端口转发——`ssh -N -L 13080:127.0.0.1:3080 KNtnx@<wsl-ip> -p 22 -i ~/.ssh/id_dsh_wsl` (图中 IP 为本机 WSL 虚拟网卡地址, 动态分配)。



图 23 SSH 验证第 3 步: curl http://127.0.0.1:13080/ 返回 DSH 令牌鉴权提示 (authentication required), 随后带 ?token=... URL (已打码) 访问成功——请求穿透隧道到达远端 DSH。

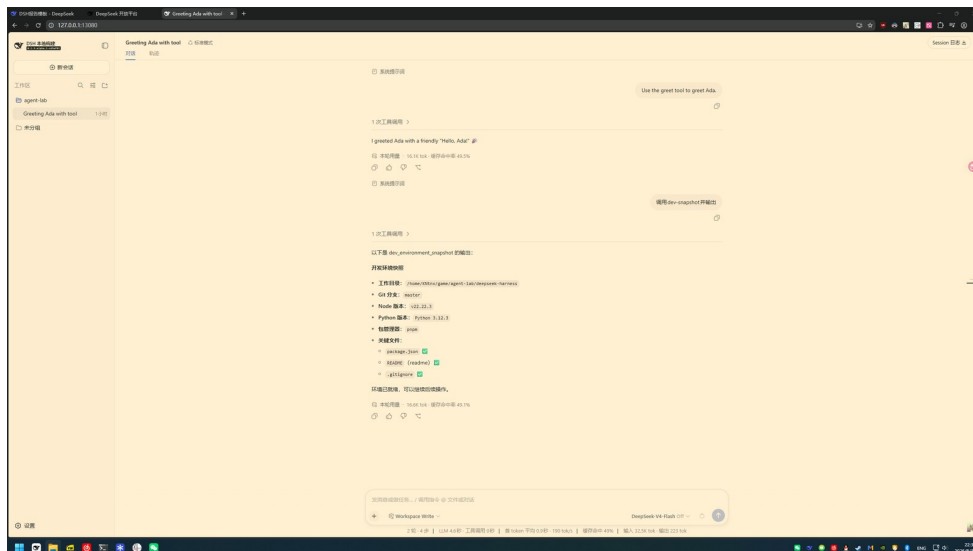


图 24 通过 SSH 端口转发访问的远端 (WSL, 独立主机视角) DSH Web UI: 地址栏 127.0.0.1:13080 (隧道本地端口), 同一会话中 greet 与 dev_environment_snapshot 两个工具被模型真实调用并展示结果。

安全要点 (对照任务书第 5 步): DSH 只监听 127.0.0.1, 从网卡不可达; SSH 转发是唯一访问通道 (加密); 公钥认证权限 600/700; DSH 令牌鉴权为第二道保护; 内网 IP 以 <ws1-ip> 占位 (动态分配 + 脱敏, 实测值仅保存在本地证据文件)。

模型配置 (第 6 步): DSH 启动时自动把环境中密钥导入 DSH_HOME/.credentials.yaml (600 权限, Web UI Settings → Models 可见); 密钥只进凭据文件与环境变量, 不进入仓库与报告。

Workspace (第 7 步): 远端项目目录 /game/agent-lab/projects 与 /game/agent-lab/deepseek-harness 在 UI “选择工作区” 可选。完整命令与本

次验证记录见 `evidence/ch4/06_remote_dsh.txt`。

4.5 问题与改进

1. **远程环境：**任务书建议远端 Linux + SSH 隧道访问 3080。本章已在本机 WSL2（Ubuntu 24.04）上以**独立远程主机**视角完整验证该方案：不依赖 WSL localhost 镜像，隧道目标为 WSL 虚拟网卡地址（`ssh -L 13080:127.0.0.1:3080 KNtnx@<wsl-ip>`，本机 3080 已被本地实例占用故映射 13080），直连登录成功、隧道端到端到达 DSH（返回鉴权 401 响应即穿透证明），完整命令与验证见 4.4.5 节与 `evidence/ch4/06_remote_dsh.txt`。端口 13080 与任务书示意的 3080 仅为本地端口号调整。注：真正的多用户远端场景（每学生一个 DSH_HOME）未实测。
2. **沙箱限制：**DSH 沙箱会拦截子进程窗口与部分 `spawn`/管道操作（`vitest` 的 `fork` 池因此不可用），测试改用 `node --test --test-isolation=none` 单进程模式；`curl` 的 TLS 凭据访问在沙箱内受限（见第 1 章环境限制说明）。
3. **版本漂移：**DSH 处于 `developer preview`，文档与 API 快速迭代。本章命令与版本均记录在案：早期 `npx` 形态 `@deepseek-ai/dsh@0.1.0-rc.6`（`npm`，实际解析安装为 `CLI 0.1.0-rc.7 / 组件 0.1.0-rc.8, cordis 4.0.1`）；本章主环境为**源码编译**（`git clone, release 0.1.2-alpha.1, commit cd5ef81`）——两种形态版本不同，恰好验证了“记录版本、不把当前行为写成永久事实”的必要性（任务书要求）。
4. **后续方向：**把 `dev_environment_snapshot` 扩展为更完整的研发环境巡检（第 5 章 `devops-scout` 插件即为强化版：增加依赖解析、测试探测、变更安全审计）；插件继续增加配置项（`settings` 服务）与 UI 卡片（`presentCall / presentResult`）接入 DSH Web。

参考文献

- [1] DeepSeek. DeepSeek Harness GitHub 仓库 [EB/OL]. <https://github.com/deepseek-ai/DeepSeek-Harness>, 访问日期 2026-08-18.
- [2] DeepSeek. Your first plugin——DSH 官方插件教程 [EB/OL]. <https://github.com/deepseek-ai/DeepSeek-Harness/blob/main/docs/your-first-plugin.md>

.com/deepseek-ai/DeepSeek-Harness/blob/master/docs/user/develop/basic/index.md, 访问日期 2026-08-18.

- [3] DeepSeek. Build a tool——DSH 官方工具教程 [EB/OL]. <https://github.com/deepseek-ai/DeepSeek-Harness/blob/master/docs/user/develop/basic/tool.md>, 访问日期 2026-08-18.
- [4] DeepSeek. Package and install a plugin——DSH 官方发布教程 [EB/OL]. <https://github.com/deepseek-ai/DeepSeek-Harness/blob/master/docs/user/develop/basic/publish.md>, 访问日期 2026-08-18.
- [5] DeepSeek. Tool authoring reference——DSH 工具契约参考 [EB/OL]. <https://github.com/deepseek-ai/DeepSeek-Harness/blob/master/docs/cookbook/adding-a-tool.md>, 访问日期 2026-08-18.
- [6] DeepSeek. DSH Architecture[EB/OL]. <https://github.com/deepseek-ai/DeepSeek-Harness/blob/master/docs/architecture.md>, 访问日期 2026-08-18.
- [7] OpenSSH. Port forwarding manual[EB/OL]. <https://www.openssh.com/manual.html>, 访问日期 2026-08-18.
- [8] Bilibili. 【极速入门】DeepSeek Harness 上手实战指南 [EB/OL]. <https://www.bilibili.com/>, 访问日期 2026-08-18.（视频资料仅作辅助；本章关键命令与事实均以官方文档与源码为准，未采用视频内容作为依据——任务书允许视频仅作参考。）

5 第五章（选作）：用 DSH 实现个人 Alpha ——项目学习报告助手

5.1 背景：问题定义

一句话问题定义：学习或接手一个陌生项目时，需要回答“它是什么、怎么搭、怎么跑”——依赖清单、文档入口、目录结构、git 状态、构建命令，人工要逐文件探测、逐个命令试跑，费时且容易漏。本 Alpha 把它做成一个 DSH 插件（code/ch5-alpha/dsh-plugins/ 仓库中的 dsh-learning-report）：用户一句话（“分析这个项目，写技术报告和复现方案”），模型按插件分发的 skill 工作流自动完成——一次收集结构化项目快照，产出 TECHNICAL_REPORT.md（技术报告）与 REPRODUCTION.md（复现方案），全部落盘为文件。

插件已安装进 **Windows 11 本机 DSH 的 desktop profile**（DSH_HOME=C:\Users\KNtnx\.dsh）。

5.2 实验目标

1. 以 bundle 形式把插件装入本机 desktop profile，并验证 **真实启动加载**（终端打印插件已加载）；
2. 梳理插件的工具/skill 结构与数据来源（文件系统与 git），画出最小架构；
3. 落实安全边界：只读收集、输出目录限定、敏感文件不读内容、人工确认；
4. 记录一次真实输入输出（运行证据，见 5.4）。

5.3 步骤与环境

环境：Windows 11（DSH_HOME C:\Users\KNtnx\.dsh，desktop profile 为 web-app 形态）；DSH 0.1.0-rc.6（npm 全局）；插件 @0.1.0（本地 bundle）。插件仓库 code/ch5-alpha/dsh-plugins/（pnpm workspaces；本 Alpha 采用项目报告插件 dsh-learning-report，其余包不在本 Alpha 范围）。安装方式与第 4 章相同（官方 bundle 流程：dsh plugin --profile desktop add <包tgz>），插件包内含 src/learning-report.ts（collect_project_context）、skills/learning-report/（项目报告五阶段工作流）与 lib/.cordis.patch.yml。**真实加载证据**见图 25（首次以默认端口启动时 EADDRINUSE: 3080，按提示改用 --port 3081 后加载成功——排错过程一并保留展示）：

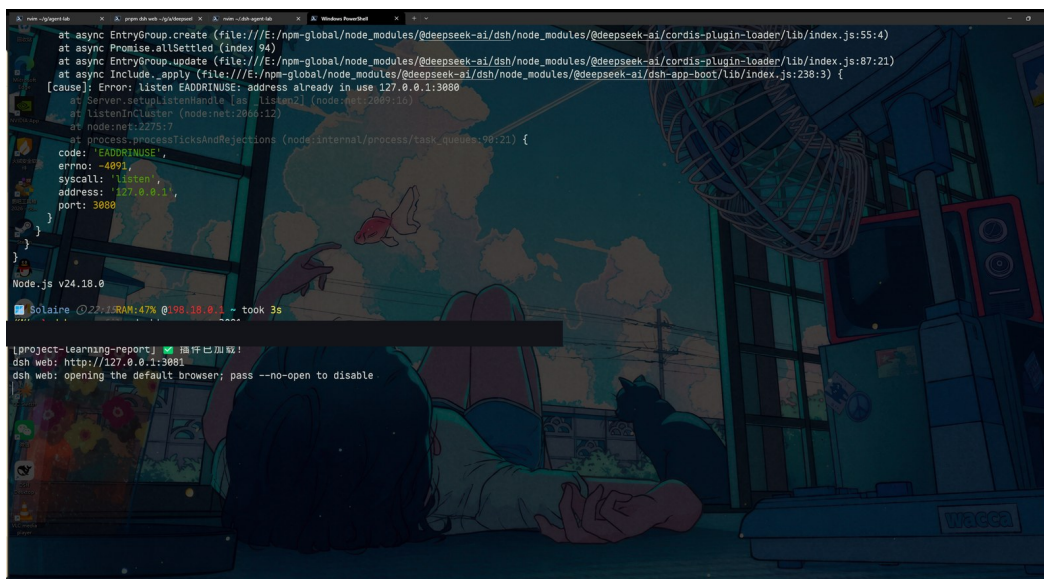


图 25 desktop profile 真实启动: `dsh --profile desktop --port 3081` 后终端打印 [project-learning-report] 插件已加载! (上方为端口占用排错记录; 图中另有一行插件加载输出已按本章范围遮盖)。

5.4 结果：真实运行证据

对真实项目 **HighWayFlow**（强化学习高速赛车 Gymnasium 环境项目，位于 `E:\game\learn-llm`）执行一次“写技术报告和复现方案”（图 26）：会话中加载 `project-learning-report` skill, `collect_project_context` 首次调用完成结构化快照收集，随后模型按 workflow 补读关键文件并落盘两份报告——会话统计共 **18 次工具调用**（1 次 `collect_project_context` + 补读/写盘等内置工具调用），随后汇报交付文件：

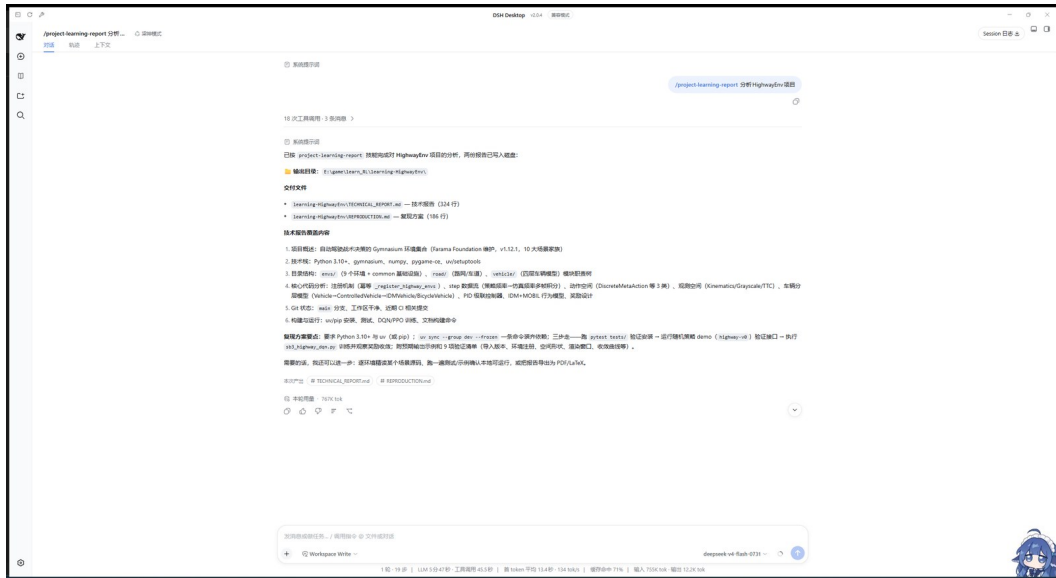


图 26 运行结果 (desktop 会话): project-learning-report 分析 HighwayFlow 项目——会话共 18 次工具调用, 输出目录 E:\game\learn-llm\learning-highwayflow, 交付 TECHNICAL_REPORT.md (328 行) 与 REPRODUCTION.md (156 行), 并给出技术报告与复现方案要点摘要。

5.4.1 交付文件 1: TECHNICAL_REPORT.md

按 SKILL 固定结构成文 (项目概述/技术栈与依赖/目录结构/核心代码分析/Git 状态/构建与运行), 图 27 为其中依赖与目录结构段 (依赖清单精确到版本与用途, 目录树带中文批注——例如 highwayflow/envs/vehicle/ 下 kinematics.py (车辆运动学) 与 controller.py (纵向/横向 PID 控制器)):

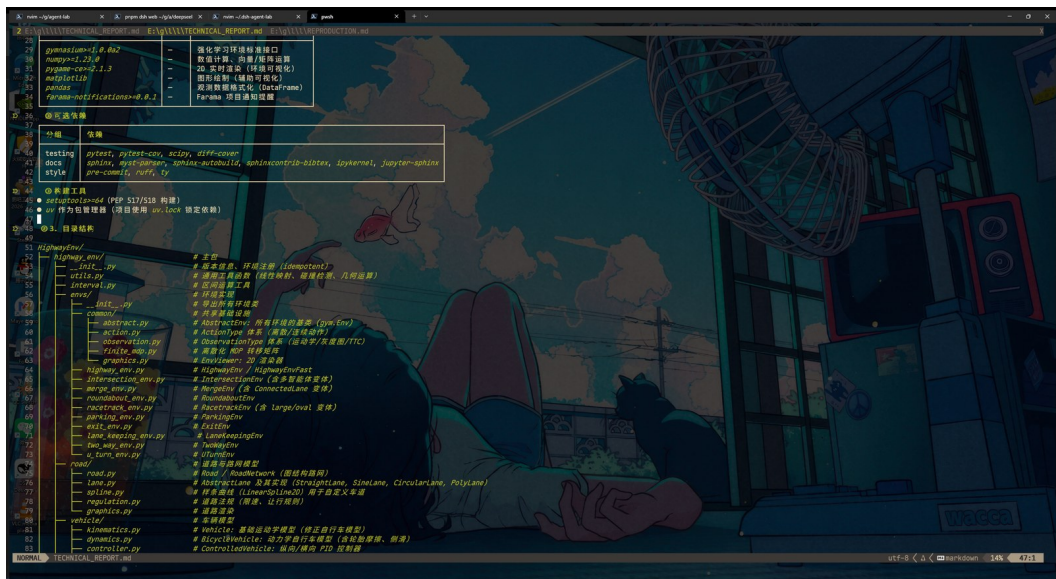


图 27 TECHNICAL_REPORT.md (编辑器打开): 第 3 段依赖与工具 (gymnasium 0.31.0、numpy、pandas 等; testing/style 工具表; uv 作为包管理器) 与第 4 段目录结构 (带中文模块批注)——高亮行为截图中的表格例。

5.4.2 交付文件 2: REPRODUCTION.md

按 SKILL 固定结构成文（环境要求/依赖安装/复现步骤/预期结果/验证清单），图 28 为第 4 段复现结果与第 5 段验证清单（42 个测试 12.3s 全部通过；验证清单逐项给出验证方法——如 `python -c "import highway_env"` 预期输出 1.22.1——与预期结果列一一对应）：

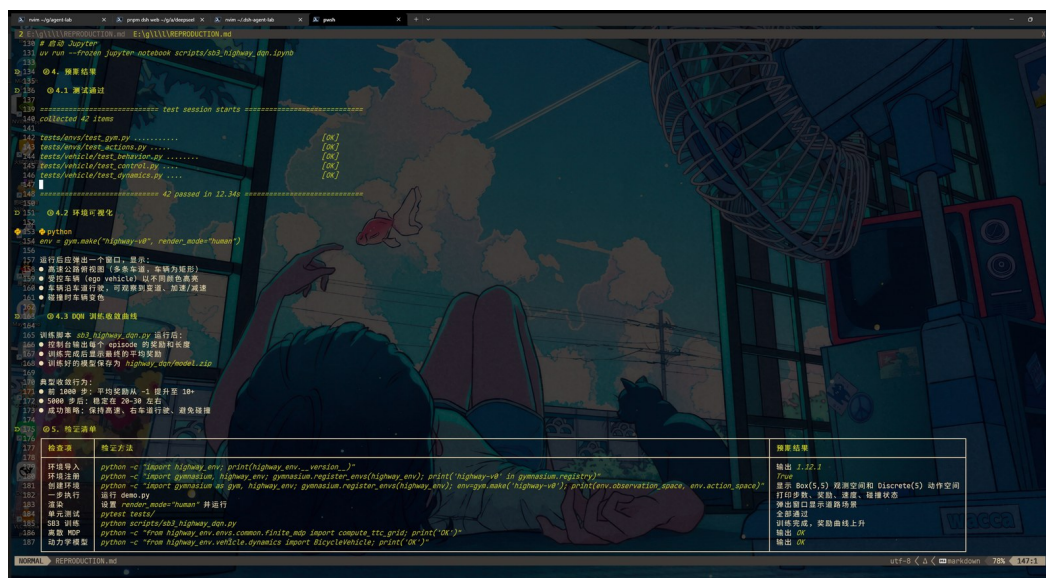


图 28 REPRODUCTION.md（编辑器打开）：第 4 段复现结果（42 tests passed in 12.3s、环境可视化、DQN 训练曲线）与第 5 段验证清单（检查项/验证方法/预期结果三列）。

至此完成“一句话 → 结构化快照 → 技术报告 + 复现方案”的完整闭环：`collect_project_context` 一次调用拿到 `manifest/README/AGENTS.md`/文件清单/git 状态（省 token），模型仅补读少数关键文件；两份报告全部落盘为文件（不在对话里贴长文）。

5.5 分析与反思

最小架构见图 29（按已安装 `bundle` 的 `cordis.patch.yml`、工具声明与 `SKILL.md` 绘制）：用户一句话 → skill 工作流阶段化驱动（阶段 1-5）→ 插件工具 `collect_project_context`（一次拿到结构化快照，省 token）与宿主内置工具（`read/write/grep/bash` 补读与落盘）→ 数据/产物两路：项目仓库（清单/文档/源码/git）⇒ 产物 `learning-< 项目 >/` 下两个 Markdown。skill 层负责“怎么编排”，工具层负责“能做什么”，分离使工具可被其它工作流复用。



图 29 个人 Alpha 最小架构：用户/会话、skill 工作流、插件工具与数据/产物四层；collect_project_context 与 project-learning-report 均来自已安装的 bundle。

为什么适合用 DSH/插件化实现：“一次收集 + 组织 + 落盘”是多工具、可组合、带权限边界的职责——DSH 插件模型统一了参数校验（defineTool）、输出 schema（供程序消费）与权限边界；skill 把收集-写作流程固化为不可变的会话行为，避免每次重复交代；bundle 可被任意 profile 复用（已验证 desktop 形态加载）。

安全边界：

1. 只读收集：collect_project_context 只读项目清单/文档/文件列表/git 状态（git 命令为只读形态），写操作限定在产物目录 learning-<项目>/；
2. 敏感文件：不读取 .env 等凭据文件（快照只含清单与文件名，不含文件内容密钥）；产物不含 API key；
3. 不覆盖：输出目录已存在时不覆盖已有报告，文件冲突采用追加或另命名（SKILL 规定）；collect_project_context 返回的清单用于快照而非逐份粘贴全文（省 token 且保证上下文可控）；
4. 人工确认：报告/复现方案为模型生成的一手产出，交付前提示用户核对关键命令与预期结果。

局限性与下一步：

1. 快照质量依赖源项目结构：manifest 识别覆盖 package.json、pyproject.toml、Cargo.toml、go.mod、requirements.txt，老式或混合项目可能识别不全；

2. `git` 依赖本机 CLI（与第 4 章相同的环境前置）；
3. 下一步：接入 `DSH tools/pre-execute` 做产出前检查；支持批量/定时学习任务；报告模板参数化。

参考文献

- [1] 本项目. `dsh-plugins`——个人插件仓库（项目学习报告插件）[EB/OL]. code/c-h5-alpha/dsh-plugins/, 访问日期 2026-08-31.
- [2] DeepSeek. DSH 技能（skills）与插件开发文档 [EB/OL]. <https://github.com/deepseek-ai/DeepSeek-Harness/tree/master/docs>, 访问日期 2026-08-31.

A 附录

A.1 代码仓库地址

- 实训工作区（完整仓库：实验脚本、证据、报告源码、插件源码，E:\game\2026 下半年暑假实训；本地 git 仓库（git init 于 2026-08-19，随章节推进持续提交，提交信息见仓库历史）；.env 已被 .gitignore 排除，不会随仓库外传。GitHub 链接：<https://github.com/KNtnx/learn-dsh>。
- 干净交付文件夹（本报告 + 任务书 + 报告用到的源码/证据/截图，无中间产物）：E:\game\2026 下半年暑假实训\2026 智能 23-1bf 陈昊 & 胡宇星组。
- 个人插件仓库（第 5 章 dsh-learning-report 等）：code/ch5-alpha/dsh-plugins/（含于工作区仓库）。
- 第 3 章分析对象：github.com/earendil-works/pi（commit 59a71b2）。
- 第 4/5 章插件参考：github.com/omdsh-dev/dsh-toolkit、github.com/omdsh-dev/dsh-tool-time 等（见各章参考文献）。

A.2 环境版本总表

表 15 全报告运行环境与依赖版本总表

项目	版本/配置
操作系统	Windows 11 专业版 (build 26200 / 25H2)
Python	3.11.15 (Miniconda 环境 ML-base, E:\Miniconda\envs\ML-base)
openai SDK	2.37.0 (第 1/2 章实验; .env 解析为脚本内置实现, 无需额外依赖)
matplotlib	3.10.9 (图表绘制)
Node.js / npm / pnpm	v24.18.0 / 12.0.2 / 11.22.0
DSH	@deepseek-ai/dsh 0.1.0-rc.6 (npm 全局)
@deepseek-ai/dsh-tools / cordis	0.1.0-rc.6 / 4.0.1
git / curl	2.55.0.windows.4 / 8.21.0
TeX Live	2025 (xelatex / latexmk / ctex / gbt7714)
Pi (第 3 章)	@earendil-works/pi-agent-core 0.84.2 (commit 59a71b2)
模型	mimo-v2.5 (第 1/2 章主实验); kimi-k3 (第 1 章模型对比实验)

A.3 参考资料总表

各章参考文献已列于章末。此处汇总全部引用类型：国内模型官方 API 文档 (Kimi/小米 MiMo/火山方舟/阿里云百炼/DeepSeek)、HTTP/JSON/curl 基础资料 (MDN/RFC 8259/curl 官方)、prompt/context/agent 论文与深度文章 (ReAct、Anthropic 系列、OpenAI harness engineering)、Pi 与 DSH 源码与官方文档、远程开发资料 (OpenSSH) 以及社区插件仓库。所有引用均注明标题、URL 与访问日期 (2026-08-17 至 2026-08-19)。

A.4 截图脱敏声明

- 第 1 章全部运行窗口截图与控制台截图经逐张检查：无 API key 明文 (平台以星号/部分字符遮挡，或由作者手动遮挡)；
- DeepSeek 控制台截图中的充值余额已由作者遮挡；累计消费为账户历史统计，图注已说明；

3. 报告全文与仓库中不存在明文密钥；`.env` 仅存本机并被 `.gitignore` 排除；
4. 第 5 章审计工具输出统一经脱敏层处理（`sk-` 等模式替换为占位符）。